EEE522 Autonomous Vehicles

Localization and Mapping (Part 2)

Ivan Ho

Acknowledgements

Part of the teaching materials in this subject was prepared based on the course F1Tenth at the University of Pennsylvania (UPenn). We would like to acknowledge Prof. Rahul Mangharam and his team for sharing the materials with us.

Part of the teaching materials in this chapter was prepared based on the online lecture of Prof. Cyrill Stachniss at the University of Bonn.

Outline

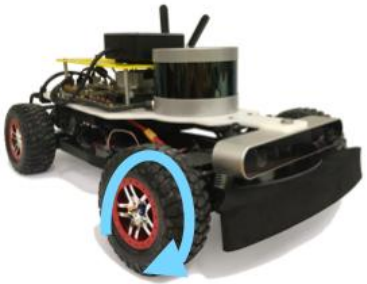
- Scan Matching
- Particle Filter
- Graph-based SLAM

Localization with Odometry

First attempt: Motion Integration

Odometry: Start at known pose and integrate control and motion measurements to estimate the current pose

- Integrate dynamics using information from VESC, wheel encoders, IMU, etc.

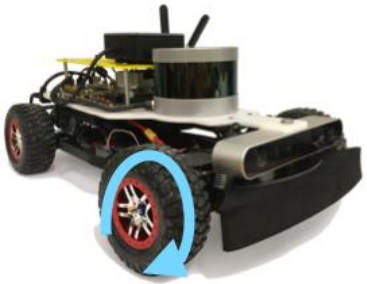


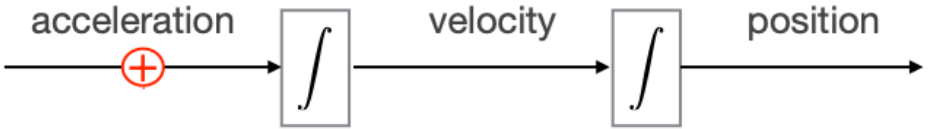
(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry: Start at known pose and integrate control and motion measurements to estimate the current pose

- Integrate dynamics using information from VESC, wheel encoders, IMU, etc.



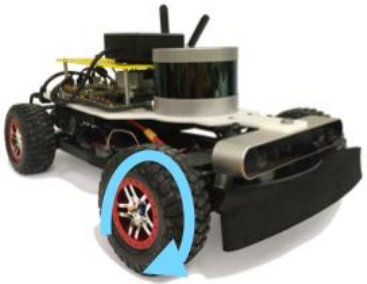


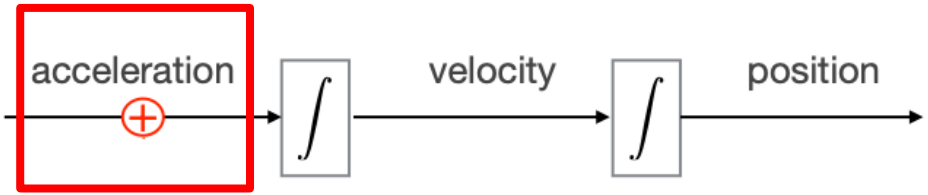
(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry: Start at known pose and integrate control and motion measurements to estimate the current pose

- Integrate dynamics using information from VESC, wheel encoders, IMU, etc.



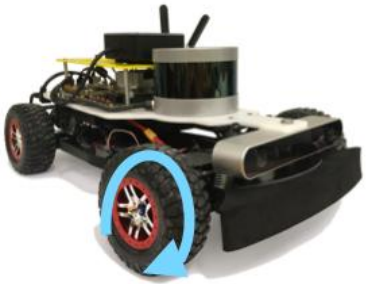


(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry: Start at known pose and integrate control and motion measurements to estimate the current pose

- Integrate dynamics using information from VESC, wheel encoders, IMU, etc.

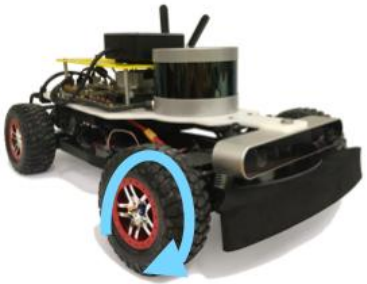


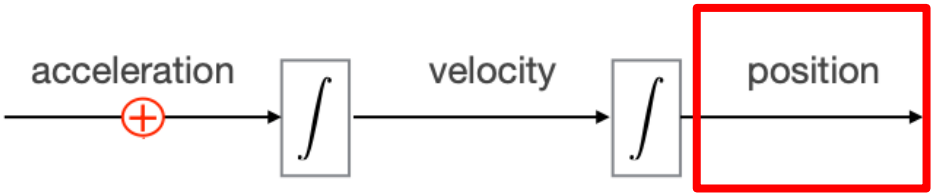
(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry: Start at known pose and integrate control and motion measurements to estimate the current pose

- Integrate dynamics using information from VESC, wheel encoders, IMU, etc.



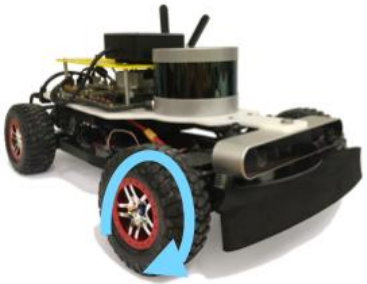


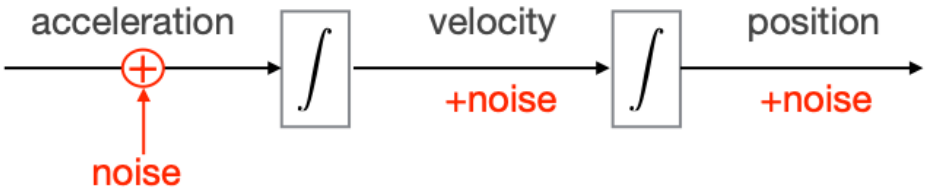
(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry: Start at known pose and integrate control and motion measurements to estimate the current pose

- Integrate dynamics using information from VESC, wheel encoders, IMU, etc.



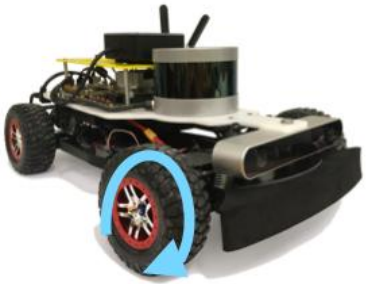


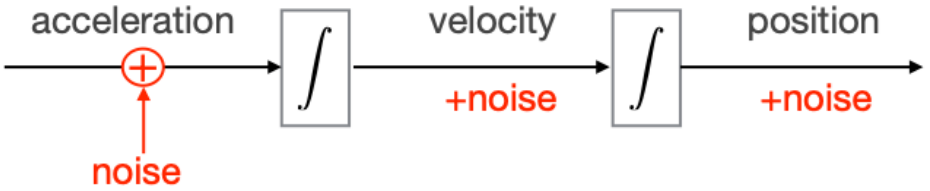
(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry: Start at known pose and integrate control and motion measurements to estimate the current pose

- Integrate dynamics using information from VESC, wheel encoders, IMU, etc.



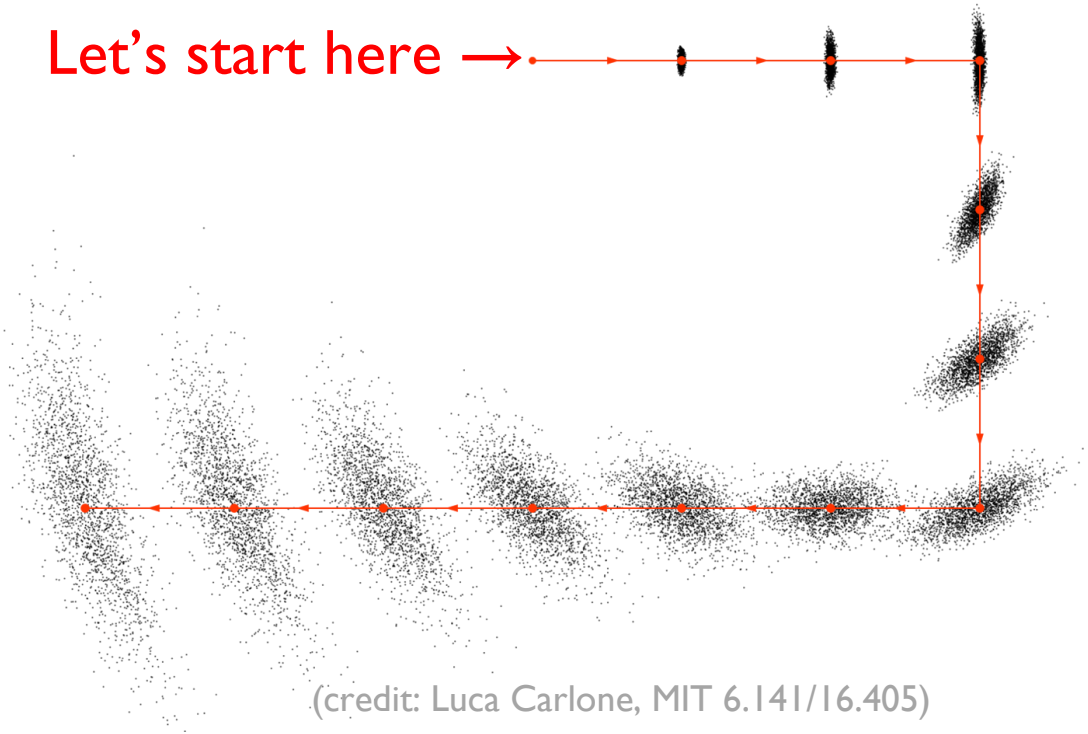


- Odometry = open loop estimation = error increases over time

(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry = open loop estimation = error increases over time

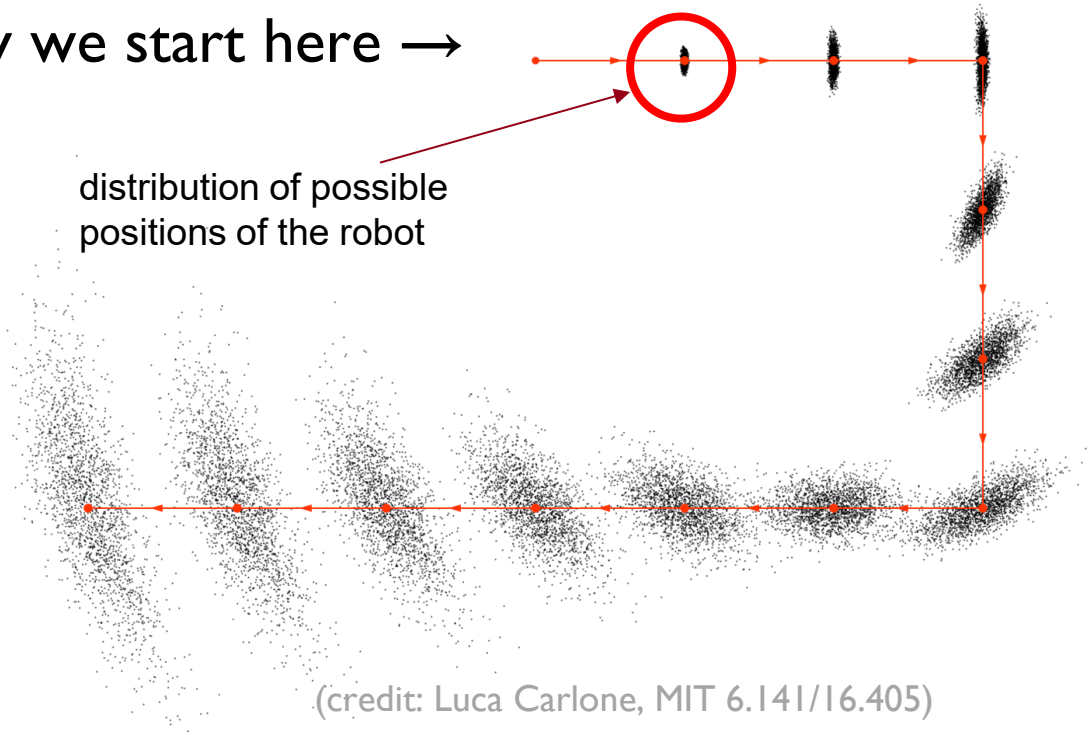


(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry = open loop estimation = error increases over time

Let's say we start here →

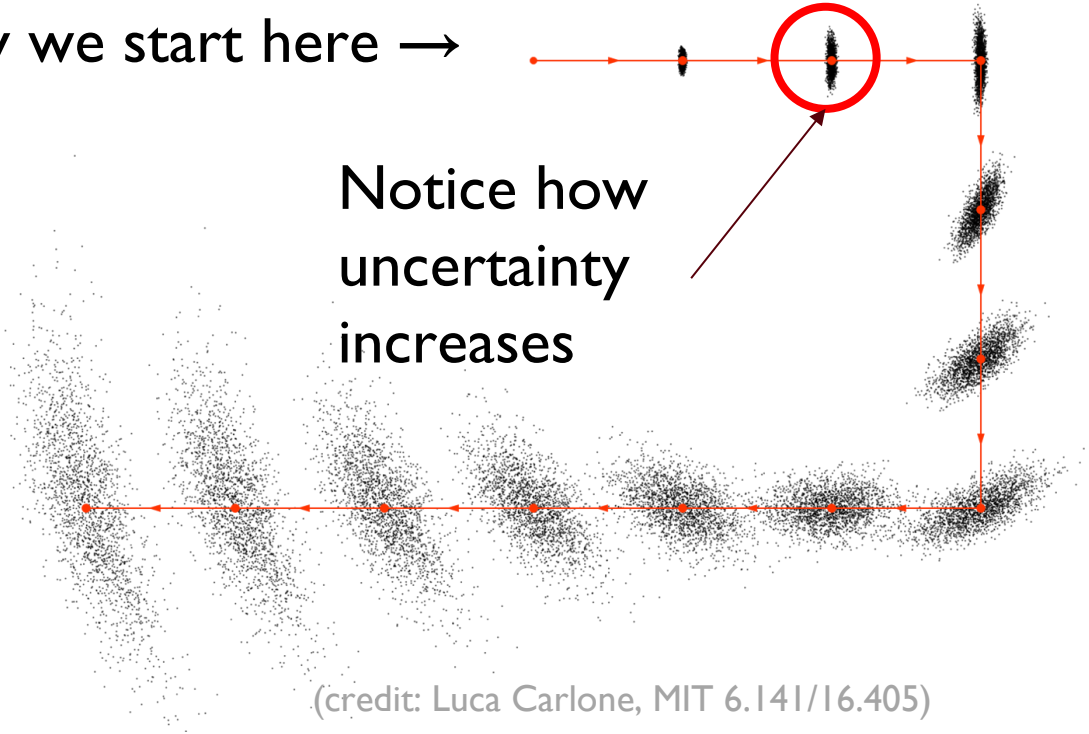


(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry = open loop estimation = error increases over time

Let's say we start here →

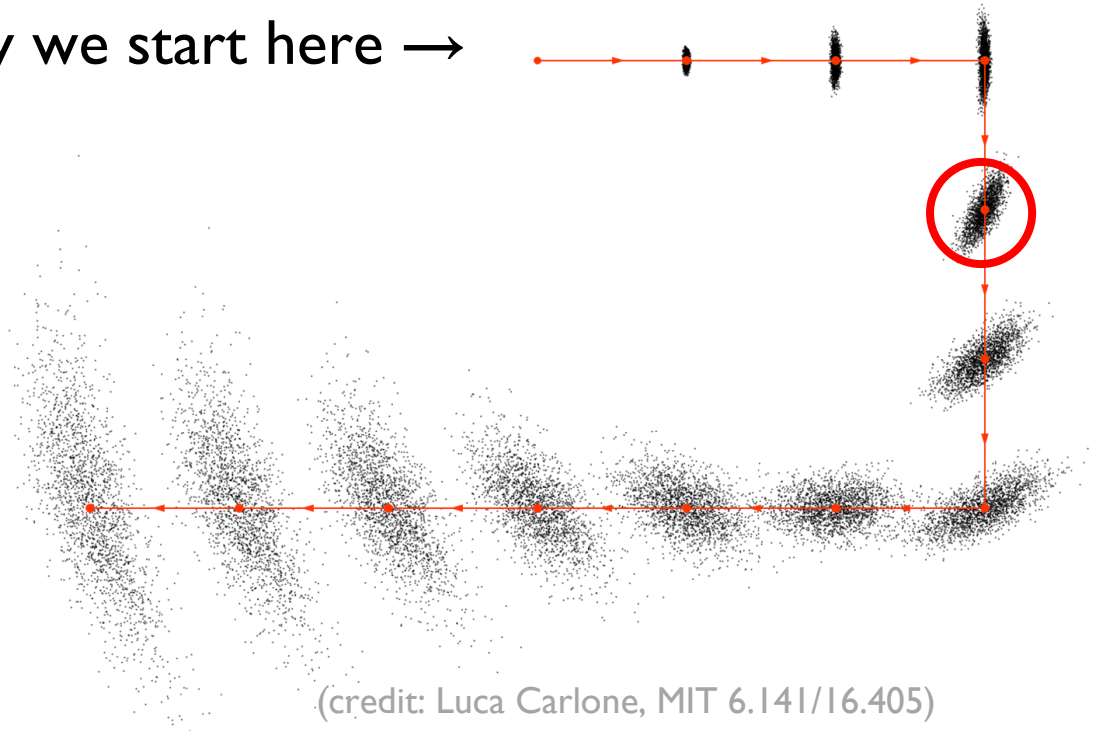


(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry = open loop estimation = error increases over time

Let's say we start here →

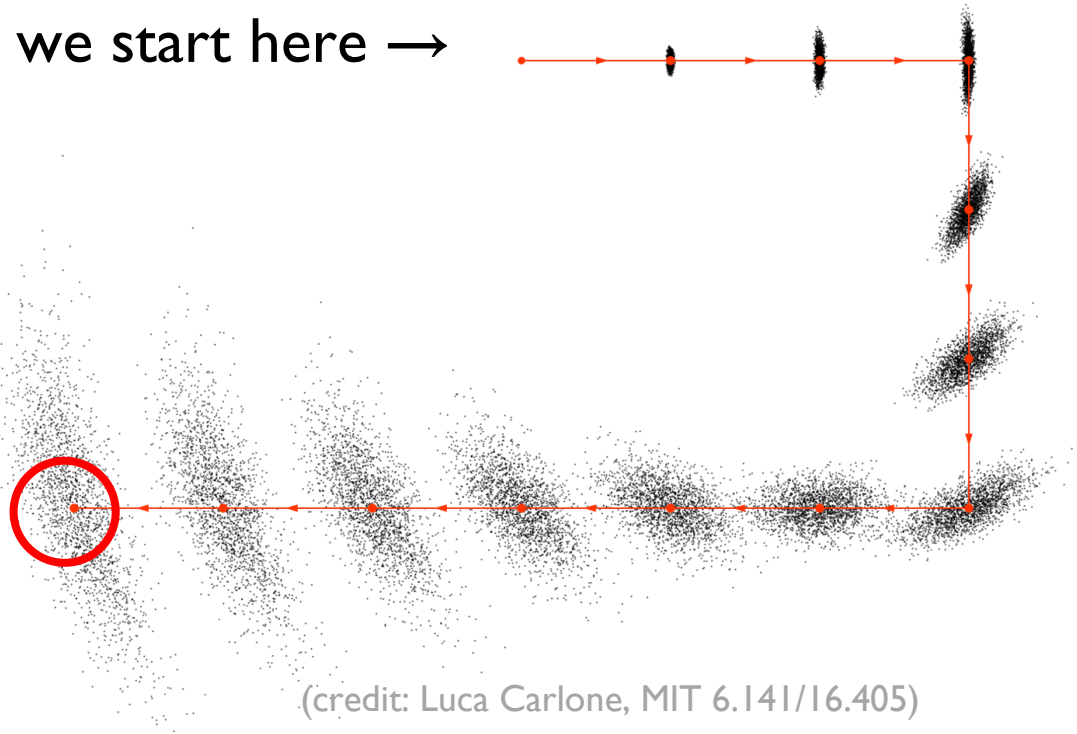


(credit: Luca Carlone, MIT 6.141/16.405)

First attempt: Motion Integration

Odometry = open loop estimation = error increases over time

Let's say we start here →



No longer able to determine correct position

(credit: Luca Carlone, MIT 6.141/16.405)

Mapping with odometry… meets reality

Open loop with no feedback

- Motion is noisy, we cannot ignore it
- Assuming known poses fails!
- Often, the sensor is rather precise

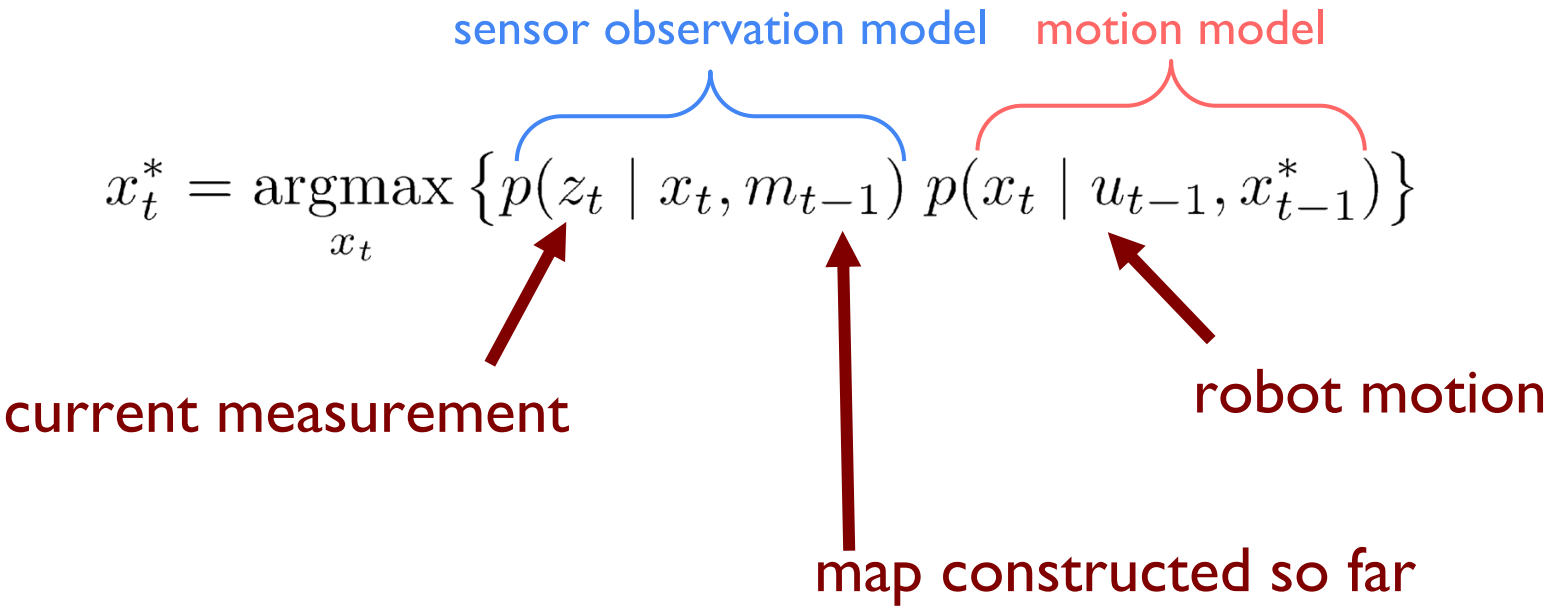
Courtesy: D. Hähnel & Cyrill Stachniss

Mapping with LiDAR

Localization using sets of range measurements collected across time, e.g. Lidar scans

Pose Correction Using Scan-Matching

Maximize the likelihood of the **current** pose relative to the **previous** pose and map

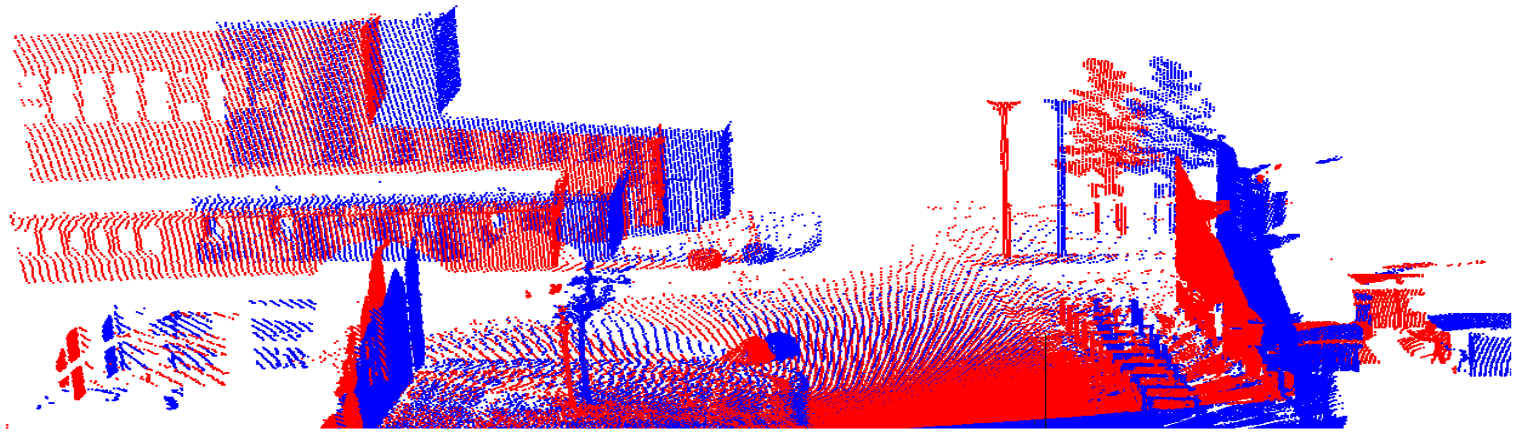


Different ways to do Scan Matching

- Iterative closest point (ICP)
- Scan-to-scan
- Scan-to-map
- Map-to-map
- Feature-based
- RANSAC for outlier rejection
- Correlative matching

Scan matching techniques use **iterative algorithms for incremental alignment** that makes grid mapping possible

Example: Aligning Two 3D Maps

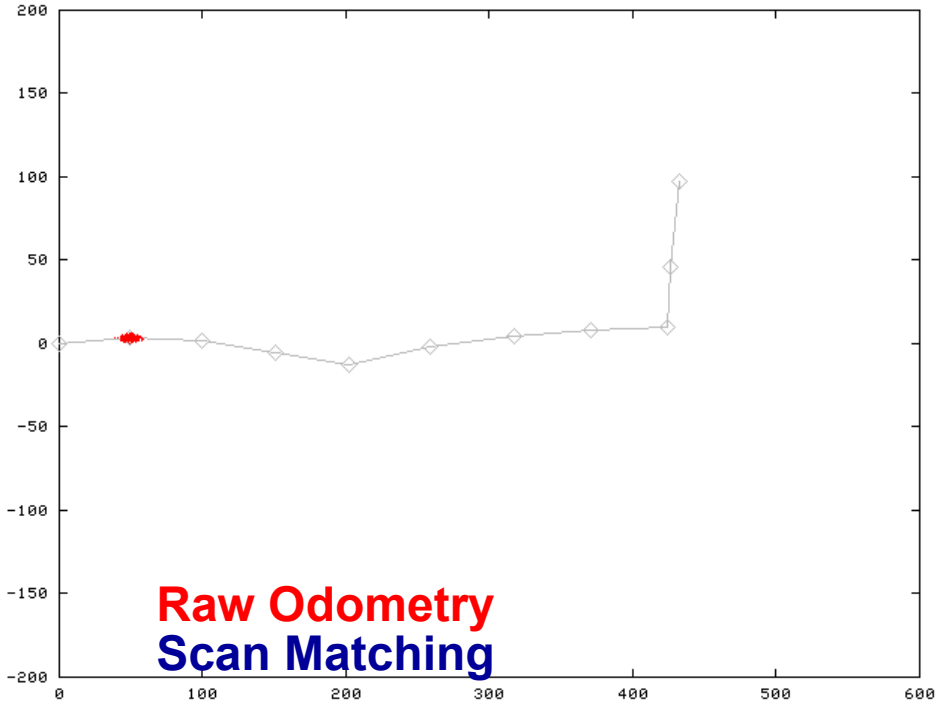


Courtesy: P. Pfaff

Mapping with LiDAR

Localization using sets of range measurements collected across time, e.g. Lidar scans

Motion Model for Scan Matching



Courtesy: D. Hähnel & Cyrill Stachniss

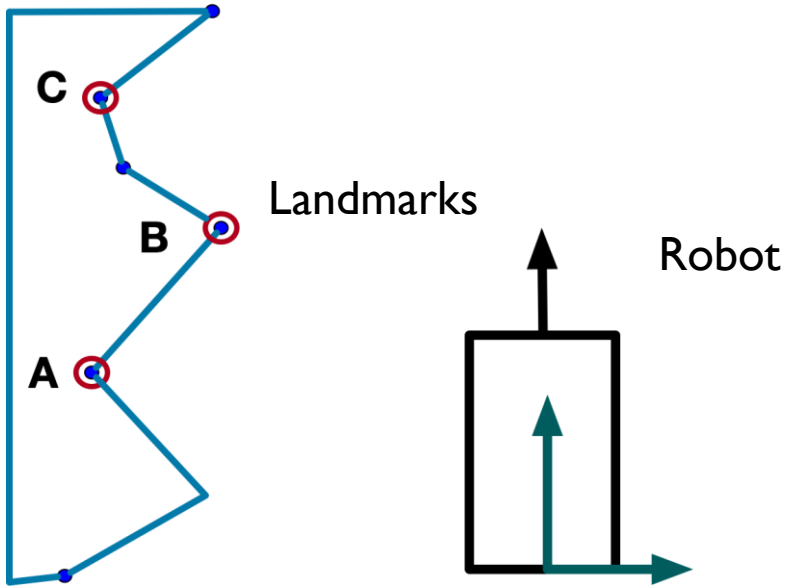
Summary of Scan Matching so far…

- Scan-matching improves the pose estimate (and thus mapping) substantially
- Locally consistent estimates
- Often scan-matching is not sufficient to build a (large) consistent map

Localization with
Iterative Closest Point Scan Matching

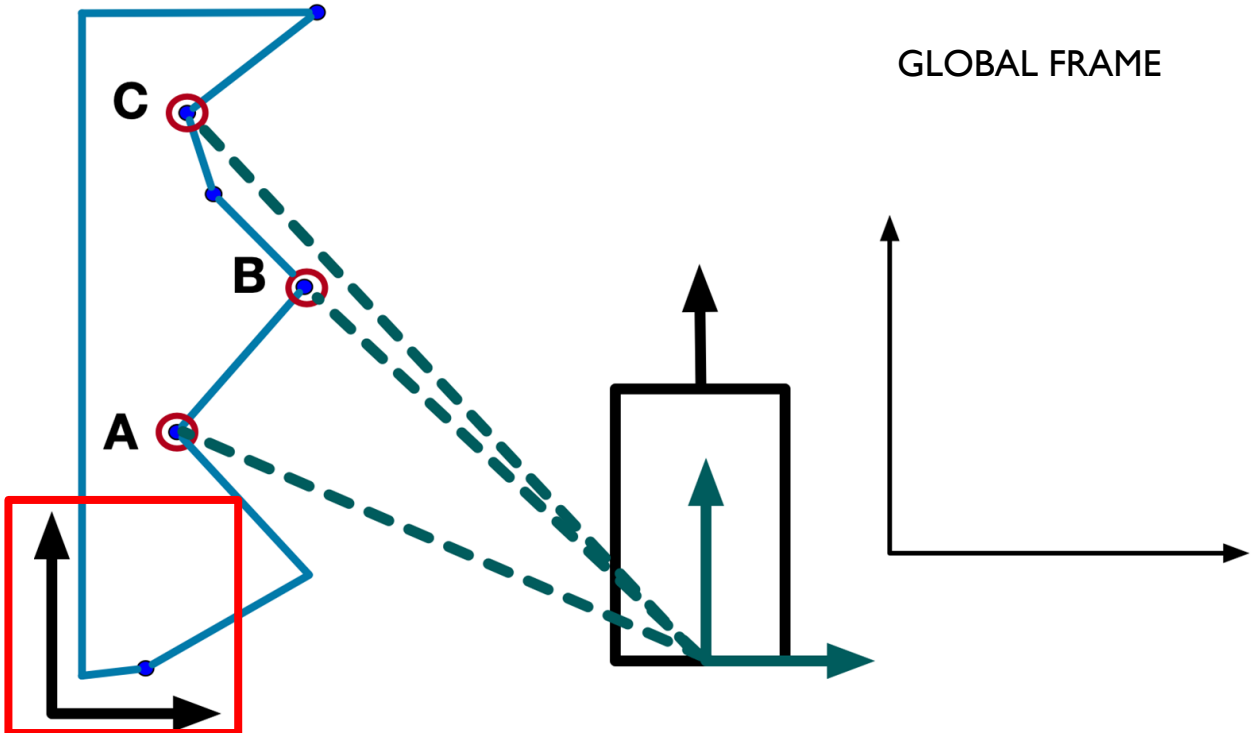
Problem Setup

Let's say my robot is in some environment with landmarks A, B, C



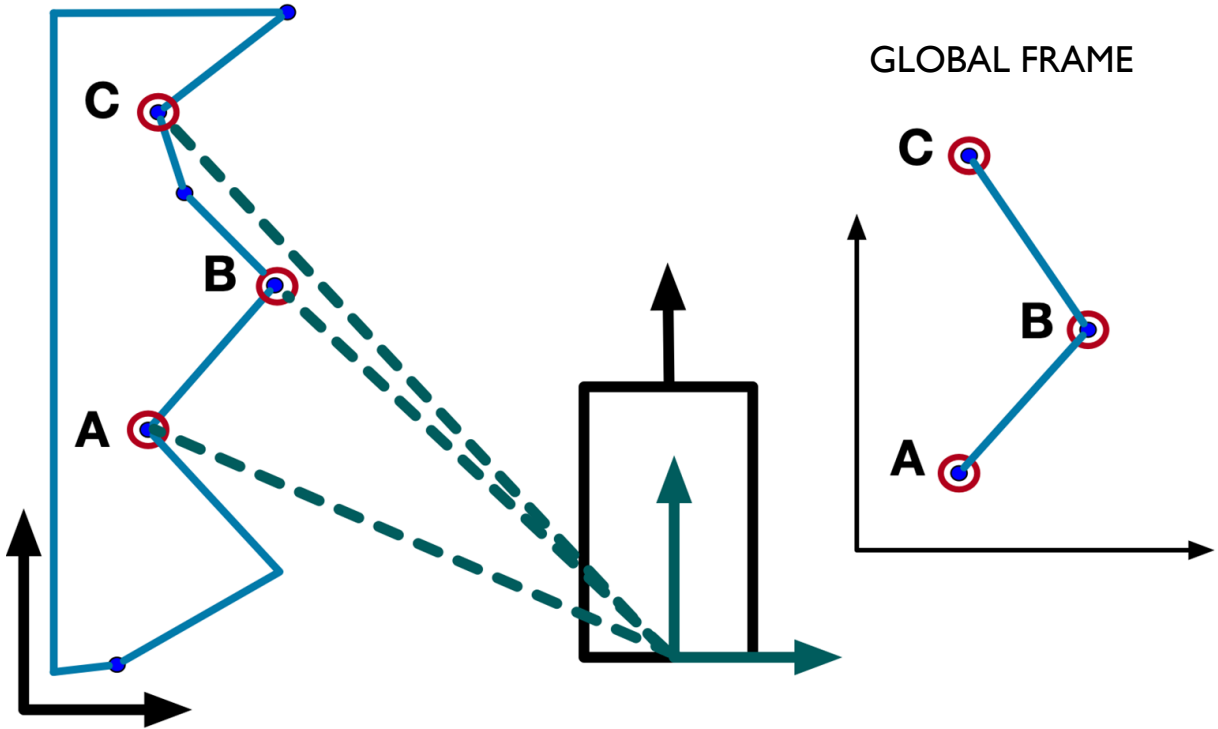
Problem Setup

At t=0, measurements of distances to A,B,C are taken:



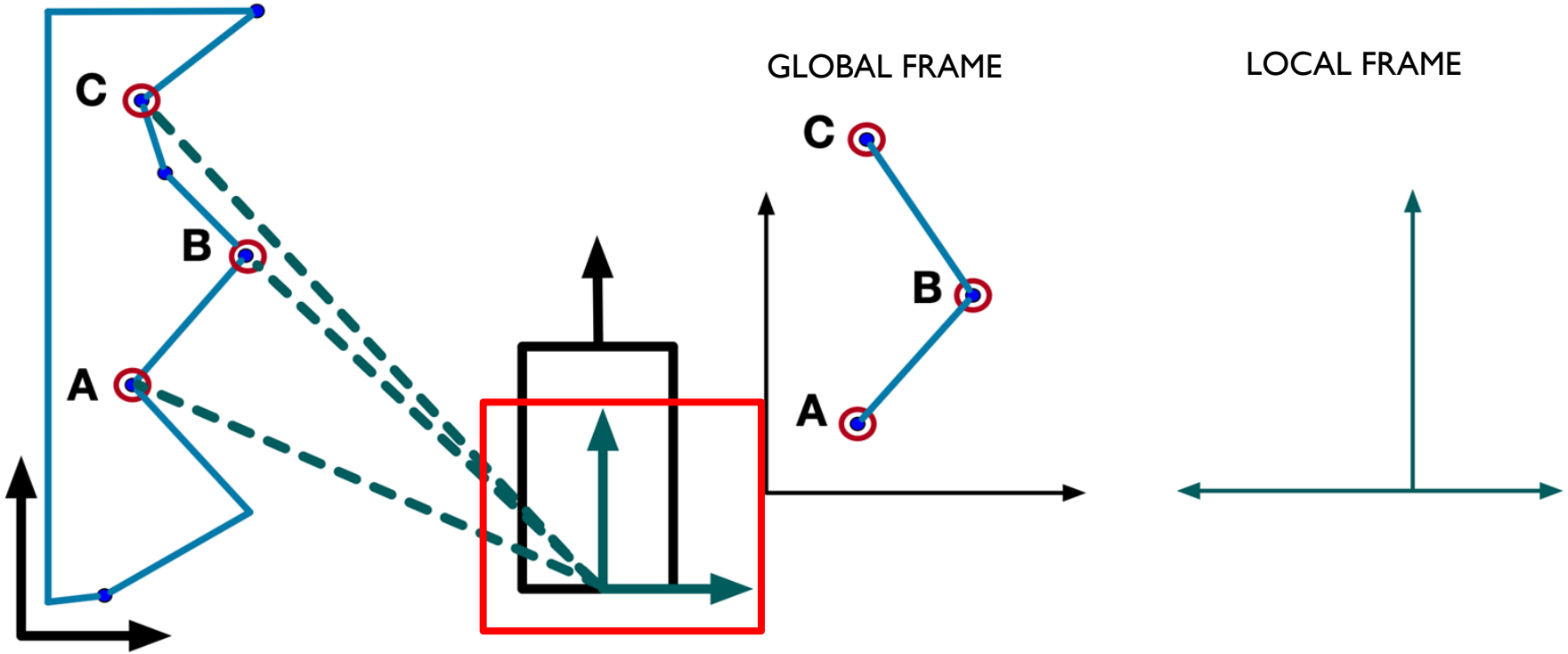
Problem Setup

At t=0, measurements of distances to A,B,C are taken:



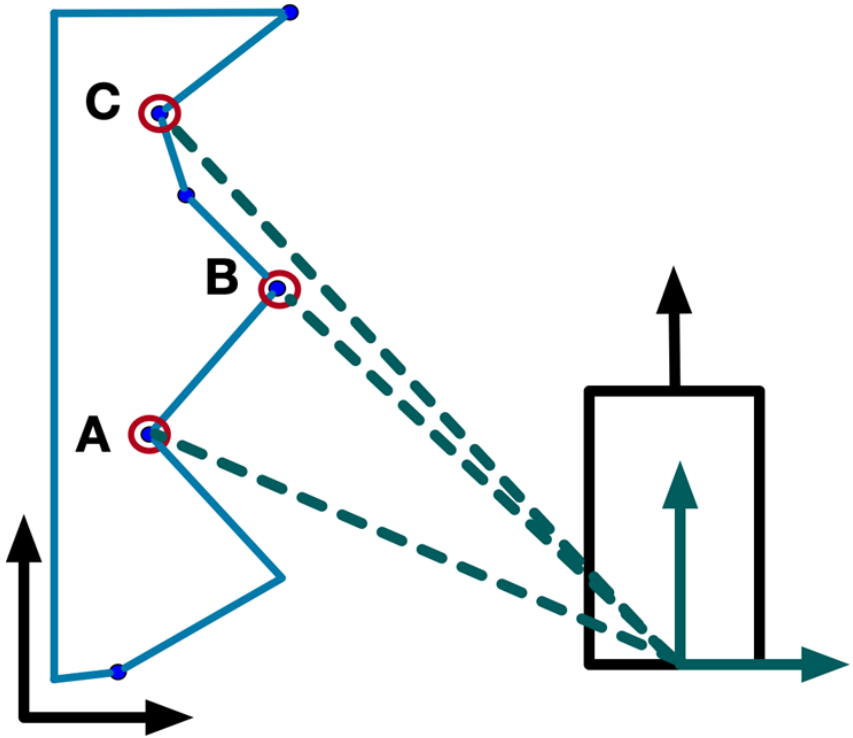
Problem Setup

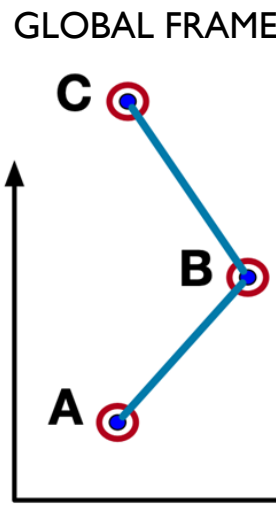
At t=0, measurements of distances to A,B,C are taken:

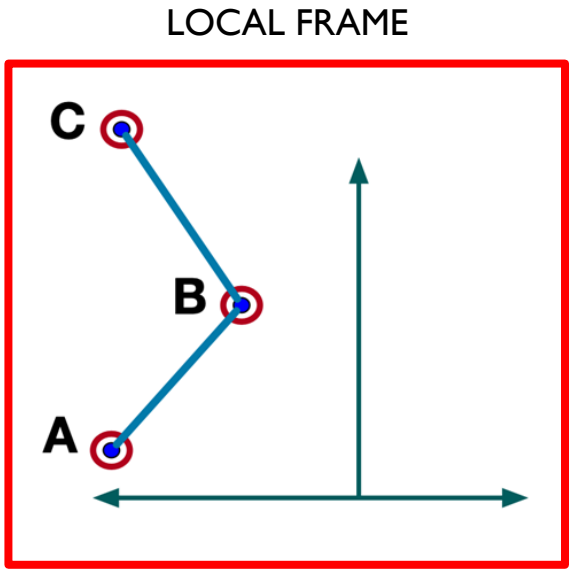


Problem Setup

At t=0, measurements of distances to A,B,C are taken:

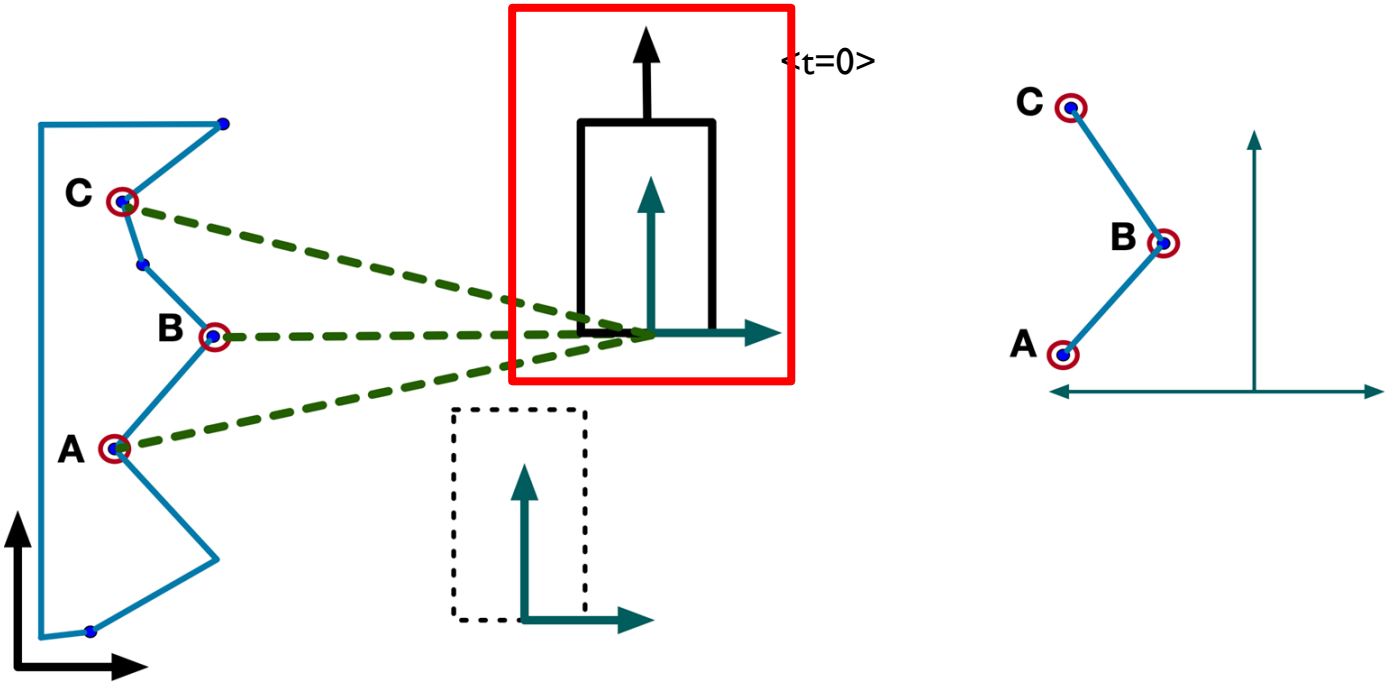






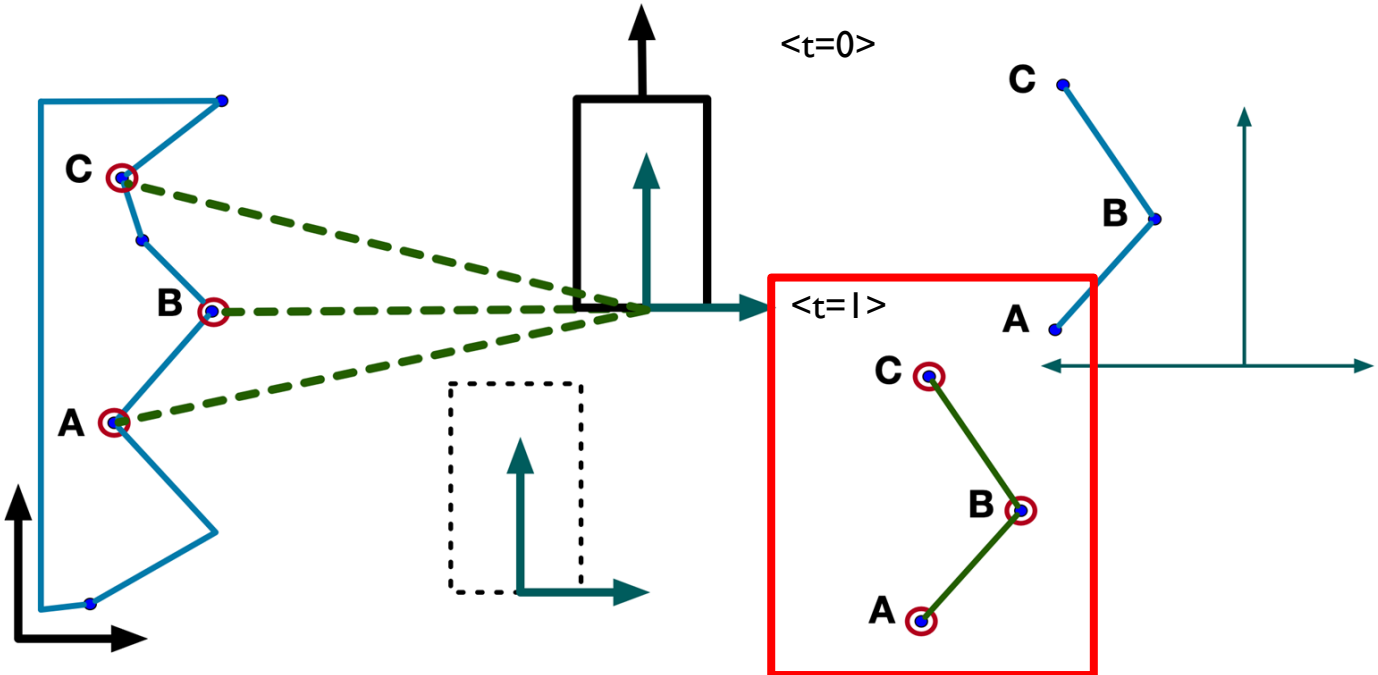
Problem Setup

At t=1, moving in some unknown direction, the distances to the landmarks will have changed:



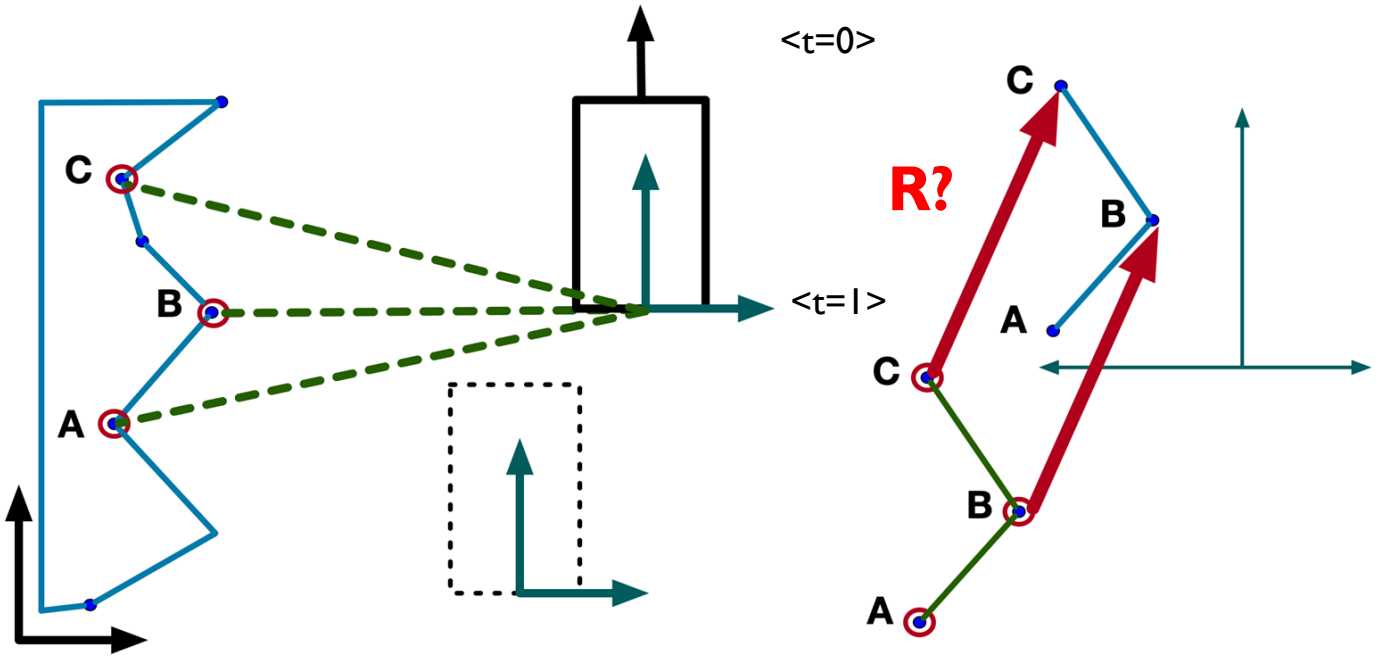
Problem Setup

At t=1, moving in some unknown direction, the distances to the landmarks will have changed:



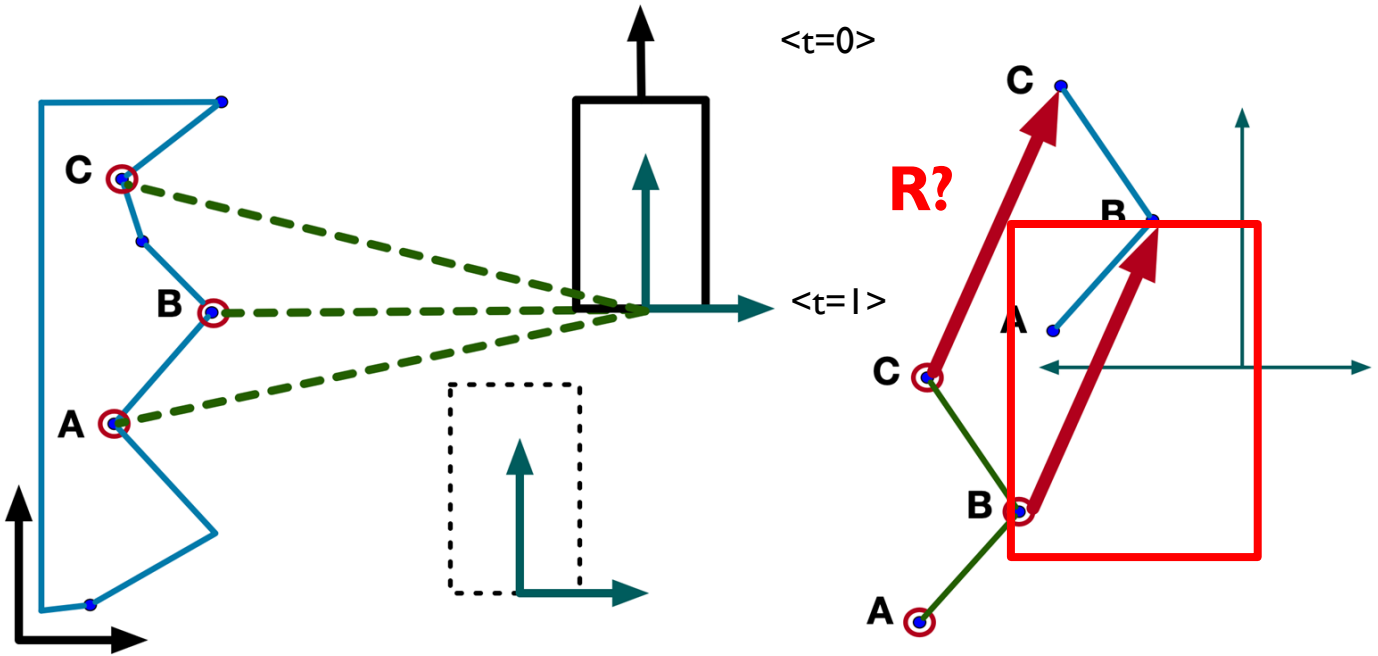
Problem Setup

We want to find transform R, that transforms the two set of points to be the closest:



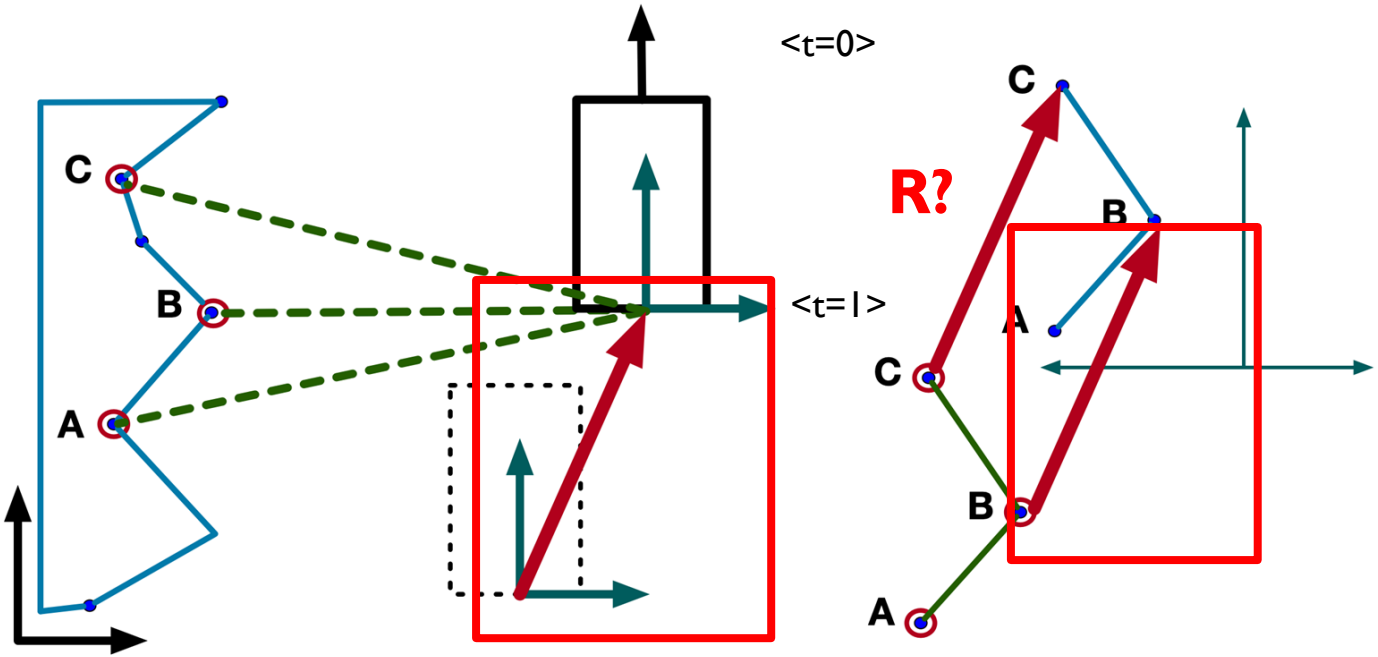
Problem Setup

Why? The transform R represents how much the robot has moved in time:



Problem Setup

Why? The transform R represents how much the robot has moved in time:



How do we find R?

Challenge:

- If we knew A, B, C (landmarks) exactly, then this would be easy
- We don't know which measurement is for which landmark

How do we find R?

Challenge:

- If we knew A, B, C (landmarks) exactly, then this would be easy
- We don't know which measurement is for which landmark

Solution:

- Assume closest points correspond to each other → "correspondence match"
- Iteratively find the best transform R

Iterative search for best transform

1. Make some initial "guess" of R (current guess)
2. For each point in new scan (t=k+1), find closest point in previous set (t=k) (**correspondence search**)
3. Make another "better guess" of R (next guess)
4. Set next guess to be the current guess, repeat steps 2-4 until converges

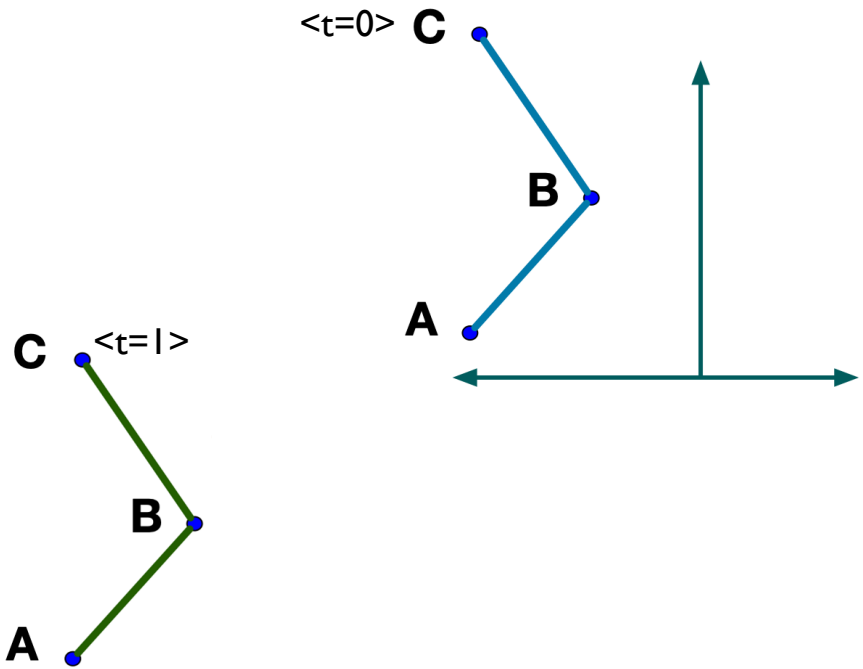
Iterative search for best transform

1. Make some initial "guess" of R (current guess)
2. For each point in new scan (t=k+1), find closest point in previous set (t=k) (**correspondence search**)
3. Make another "better guess" of R (next guess)
4. Set next guess to be the current guess, repeat steps 2-4 until converges

What makes a "better guess"? → Need a metric (match function)

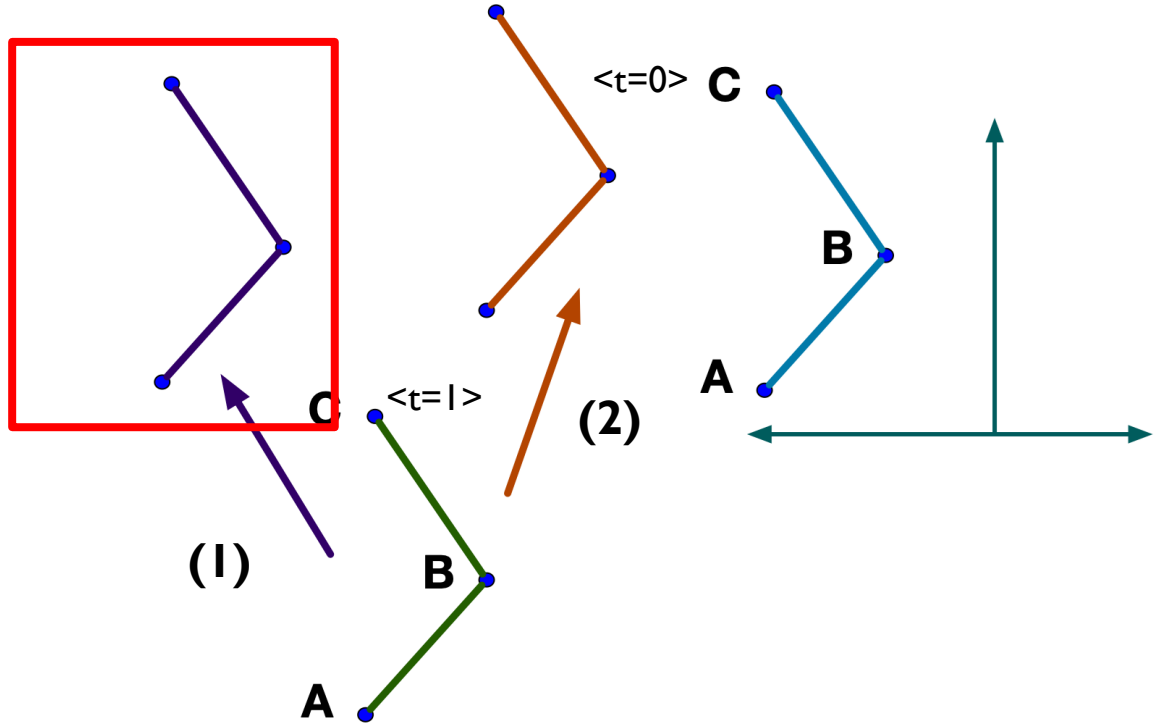
How do you choose between I or II?

(Assume correspondences have been found)



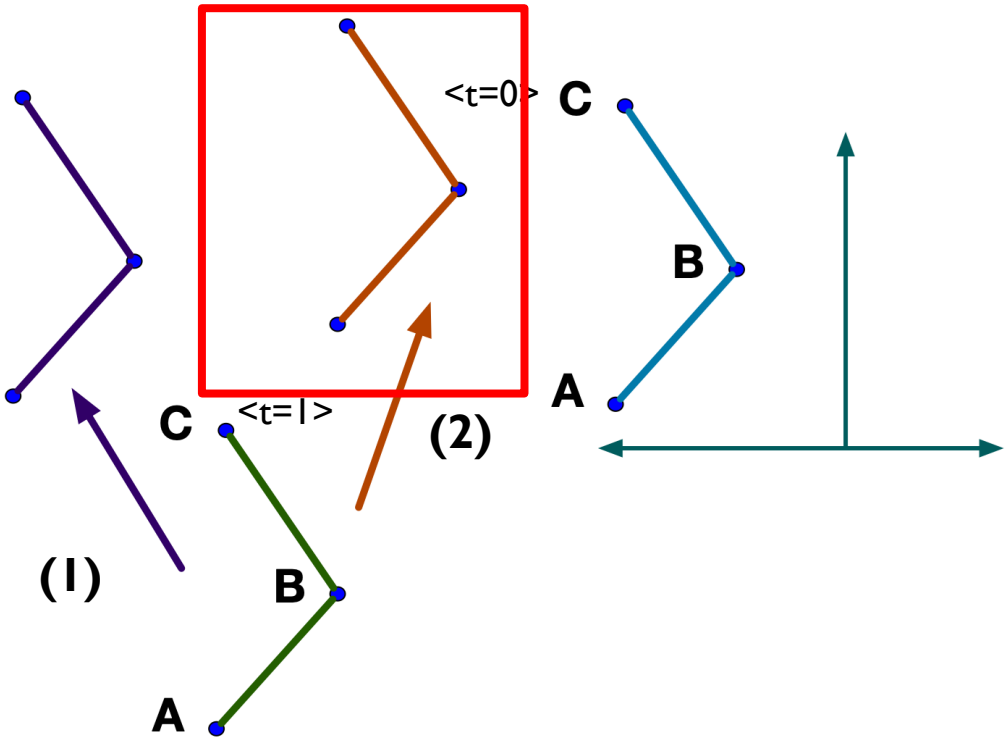
How do you choose between I or II?

(Assume correspondences have been found)



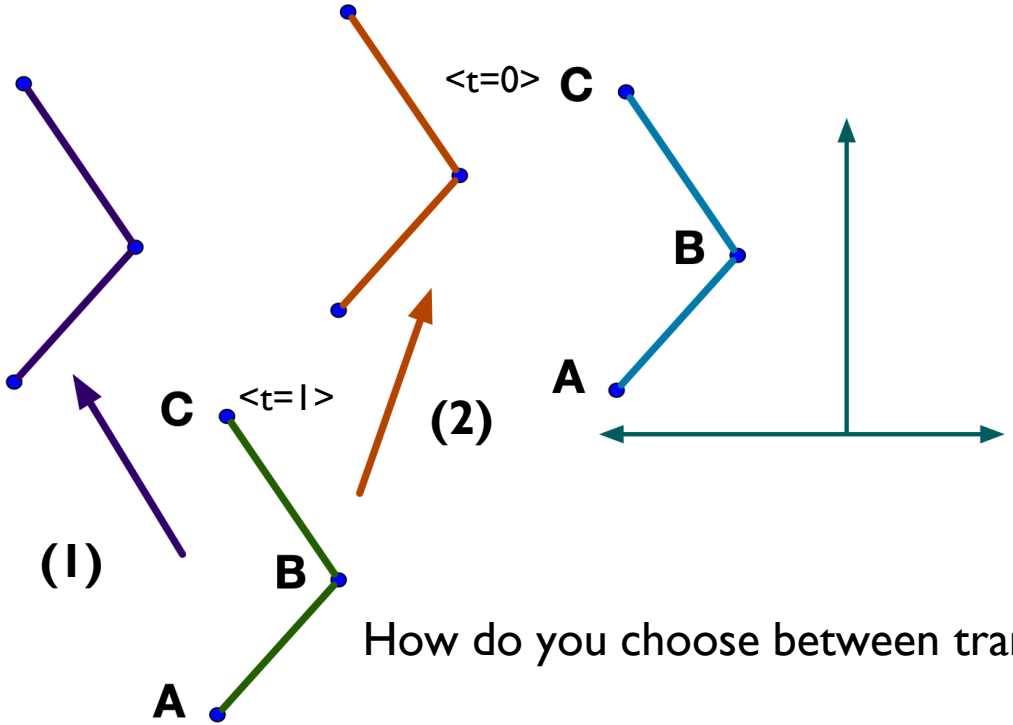
How do you choose between I or II?

(Assume correspondences have been found)



How do you choose between I or II?

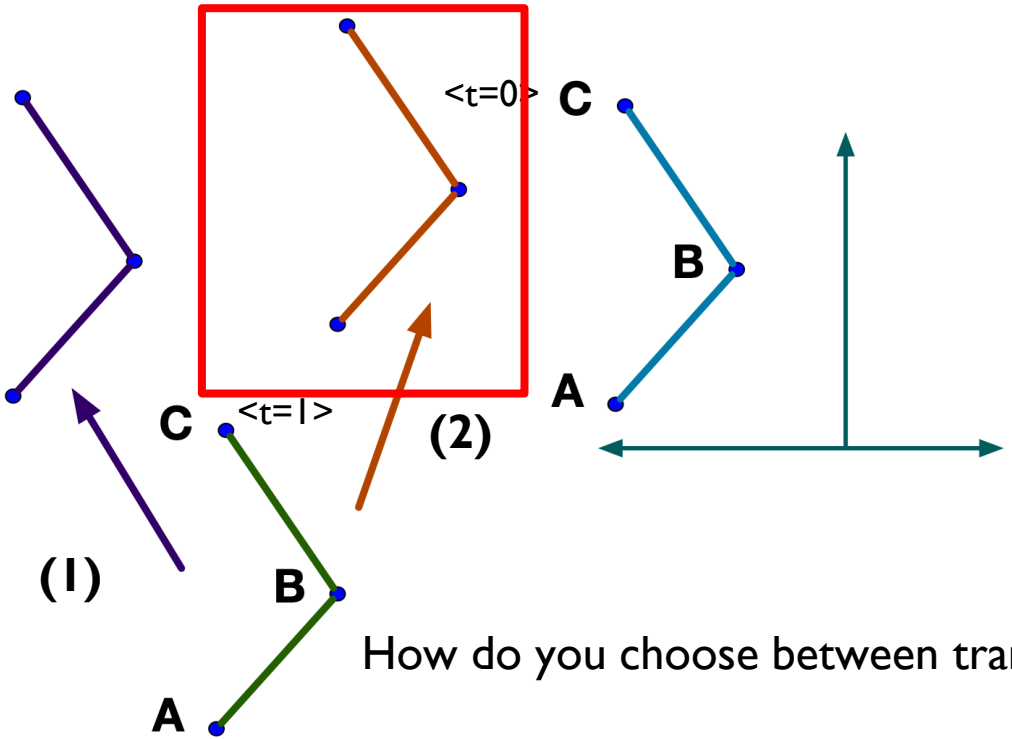
(Assume correspondences have been found)



How do you choose between transform 1 or transform 2?

How do you choose between I or II?

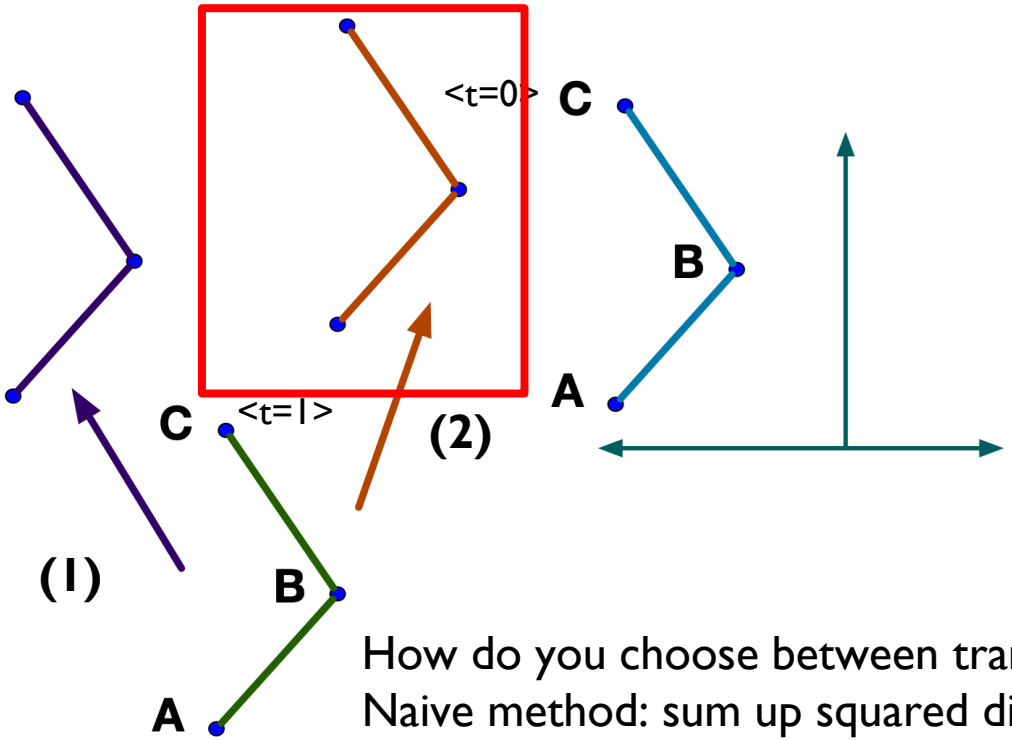
(Assume correspondences have been found)



How do you choose between transform 1 or transform 2?

How do you choose between I or II?

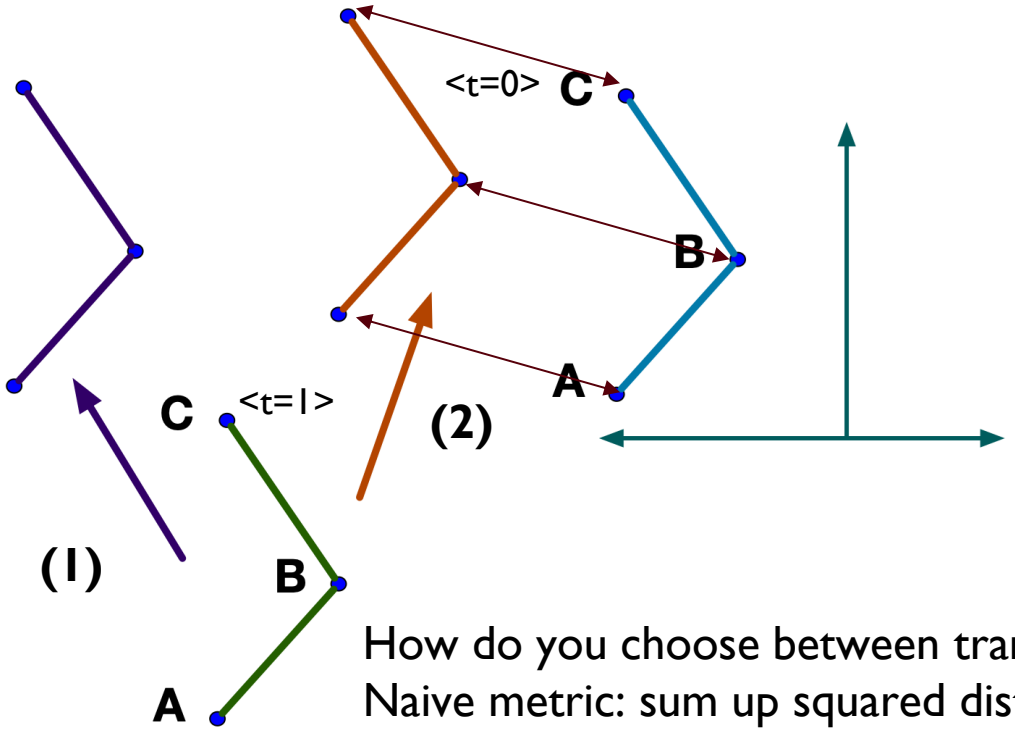
(Assume correspondences have been found)



How do you choose between transform 1 or transform 2?
Naive method: sum up squared distances

How do you choose between I or II?

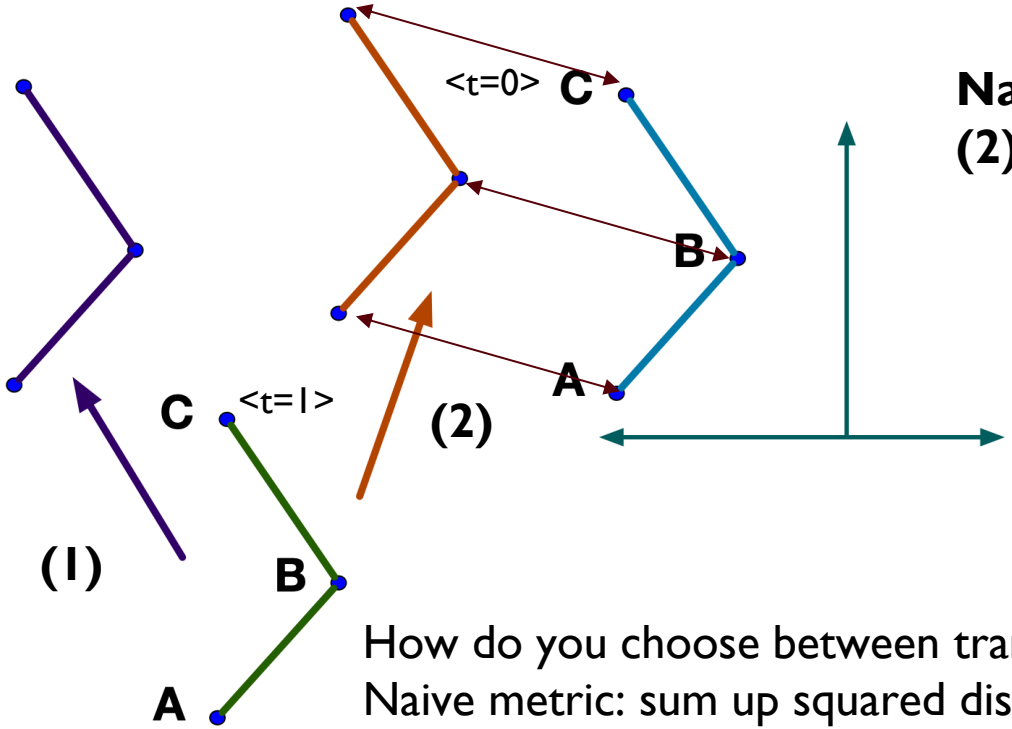
(Assume correspondences have been found)



How do you choose between transform 1 or transform 2?
Naive metric: sum up squared distances

How do you choose between I or II?

(Assume correspondences have been found)

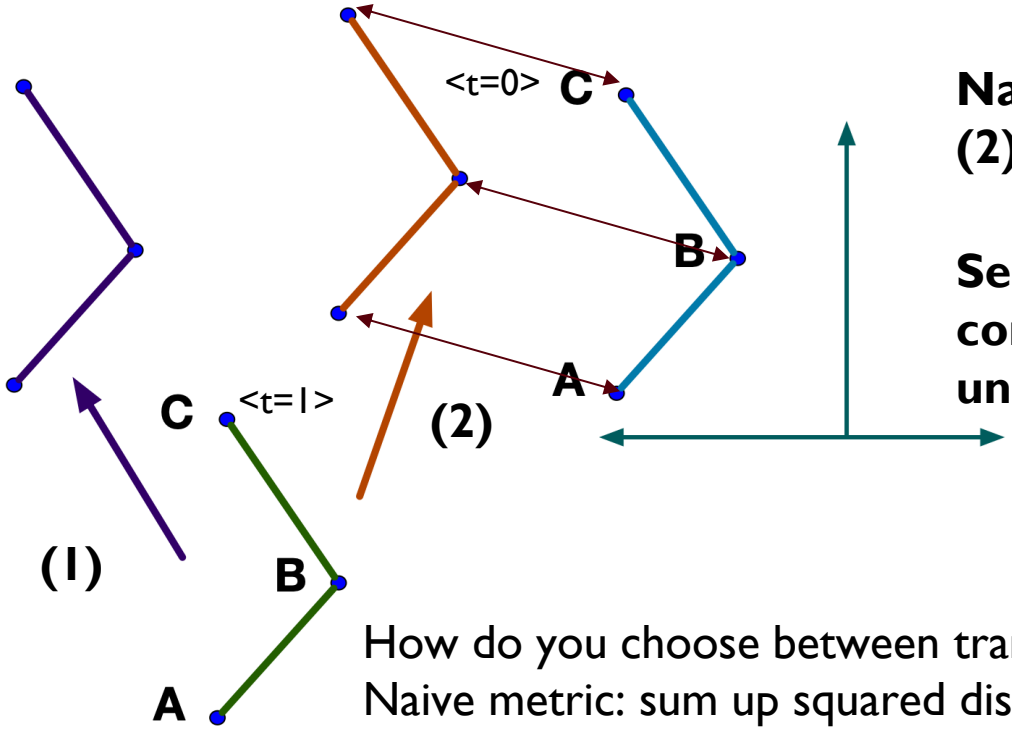


Naive metric: (2) < (1)
(2) is better

How do you choose between transform 1 or transform 2?
Naive metric: sum up squared distances

How do you choose between I or II?

(Assume correspondences have been found)



Naive metric: (2) < (1)
(2) is better

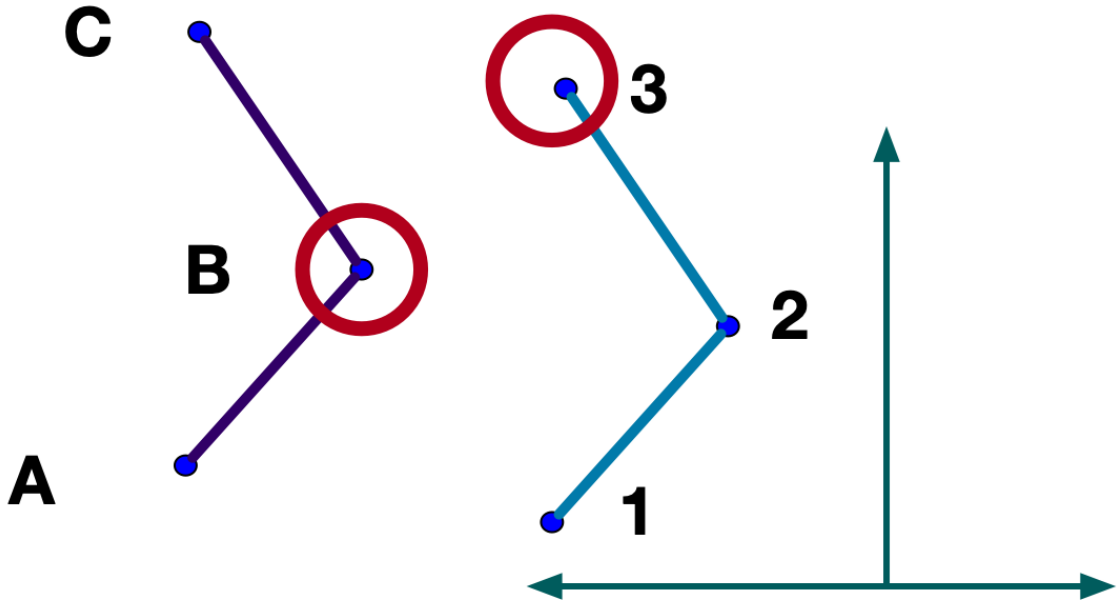
Sensitive to noise when correspondences are unknown

How do you choose between transform 1 or transform 2?
Naive metric: sum up squared distances

Better Candidate Match Function

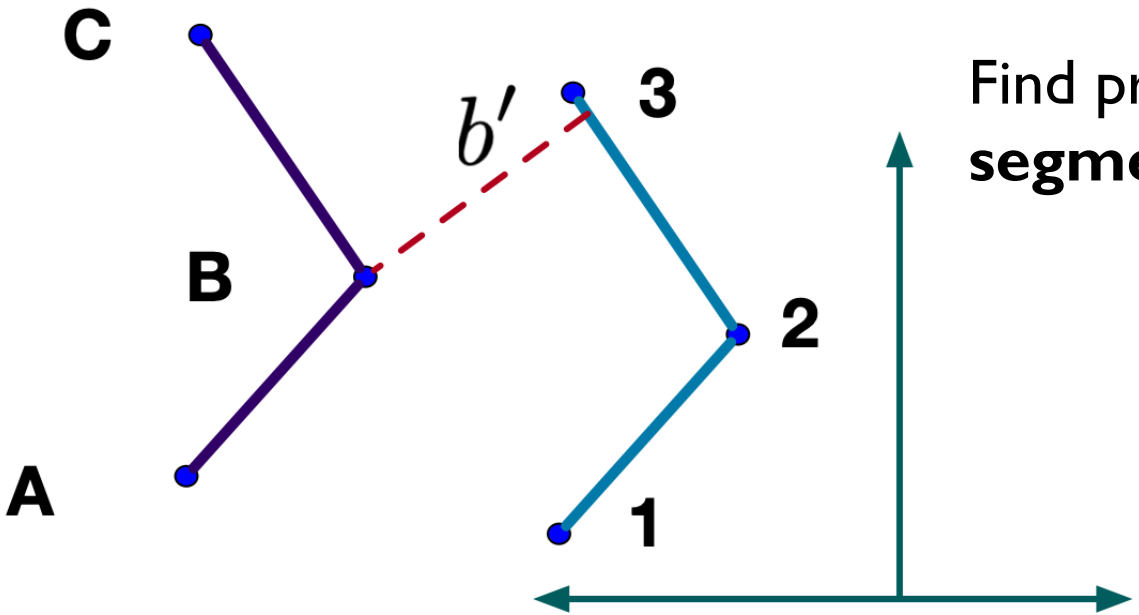
Point to projected point (point-to-point) metric:

Closest point to B is 3



Better Candidate Match Function

Find projection of point onto reference surface:

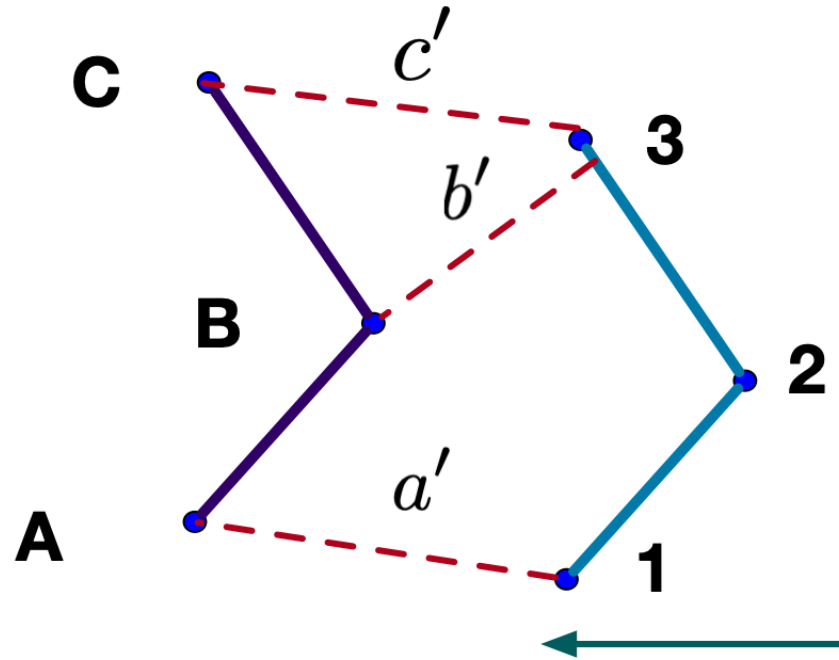


Closest point to B is 3

Find projected **point** on **line segment** 2-3, distance is b’

Better Candidate Match Function

Squared sum of projected distances (point-to-point)



Closest point to B is 3

Find projected **point** on line segment 2-3, b'

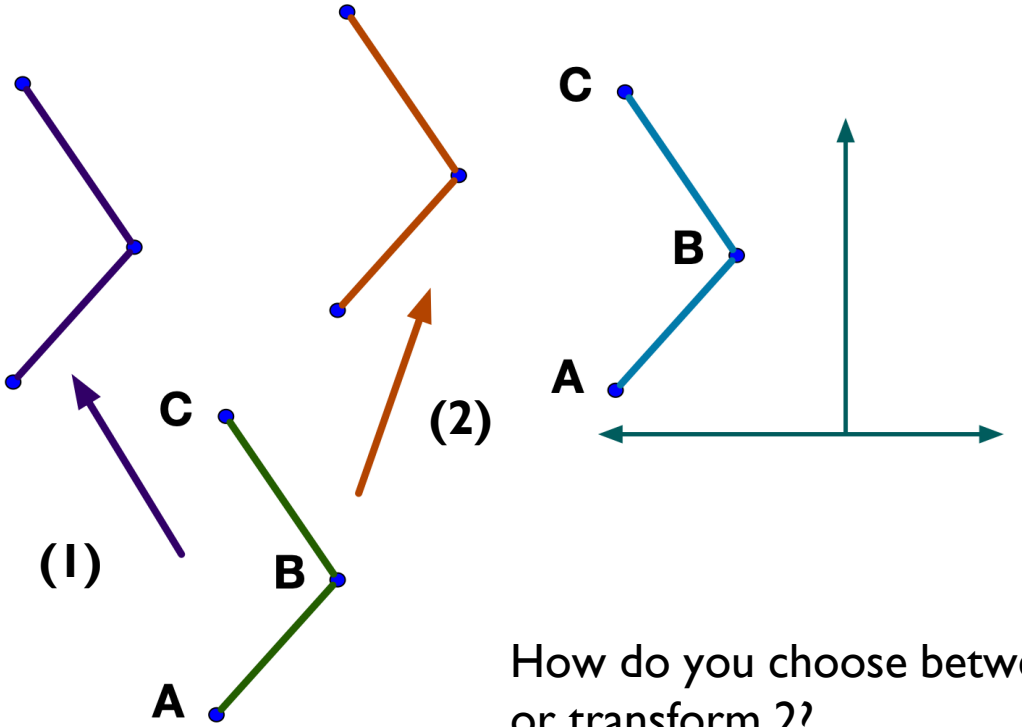
Square the sum of projected distances

$$Score = |a'|^2 + |b'|^2 + |c'|^2$$

How do you choose between I or II

Choose (2)

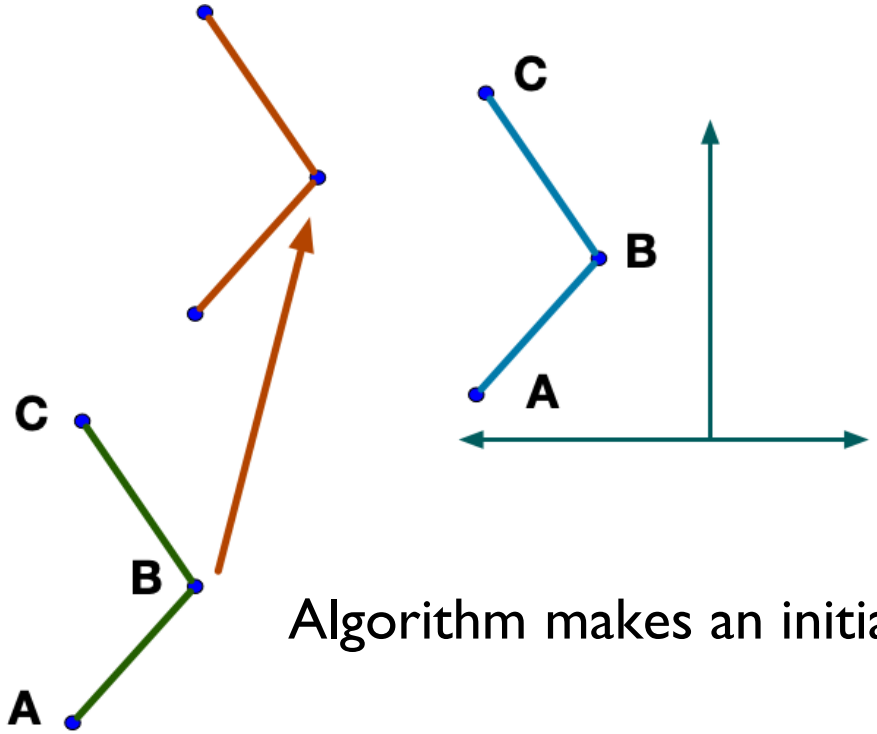
Score (1) > **Score (2)**



How do you choose between transform 1 or transform 2?

Vanilla Iterative Closest Point (ICP)

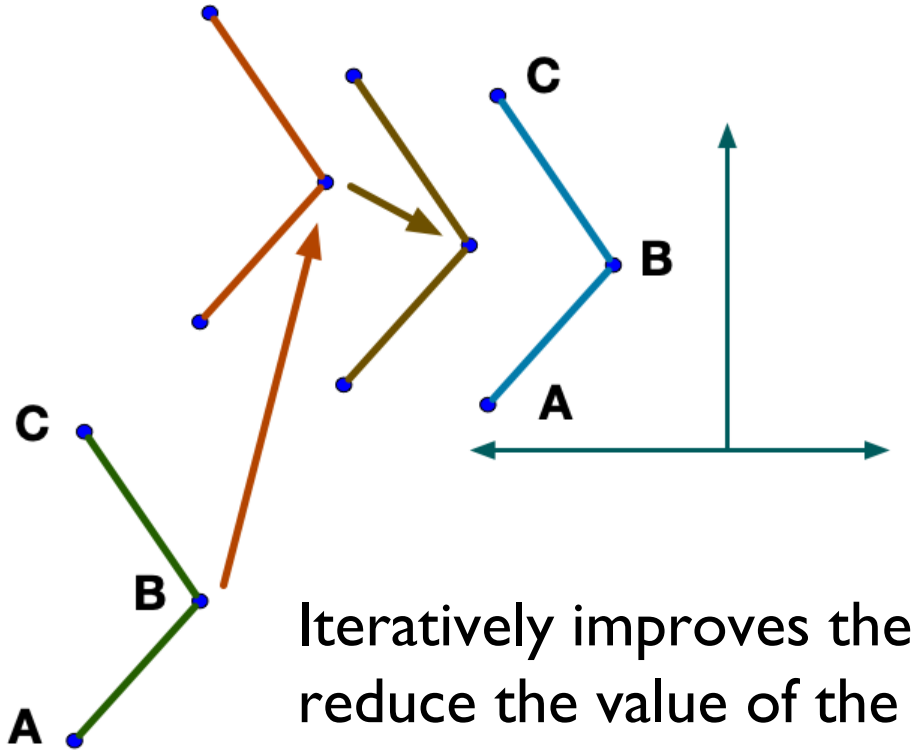
Uses point-to-point metric



Algorithm makes an initial guess of transform

Vanilla Iterative Closest Point (ICP)

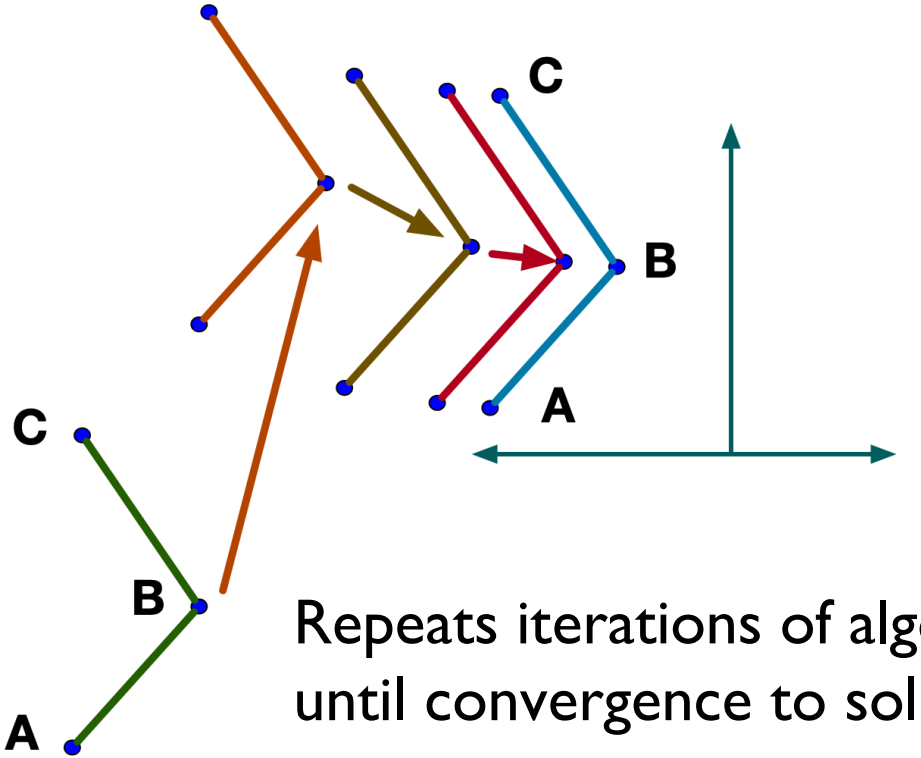
Uses point-to-point metric



Iteratively improves the guess to reduce the value of the metric

Vanilla Iterative Closest Point (ICP)

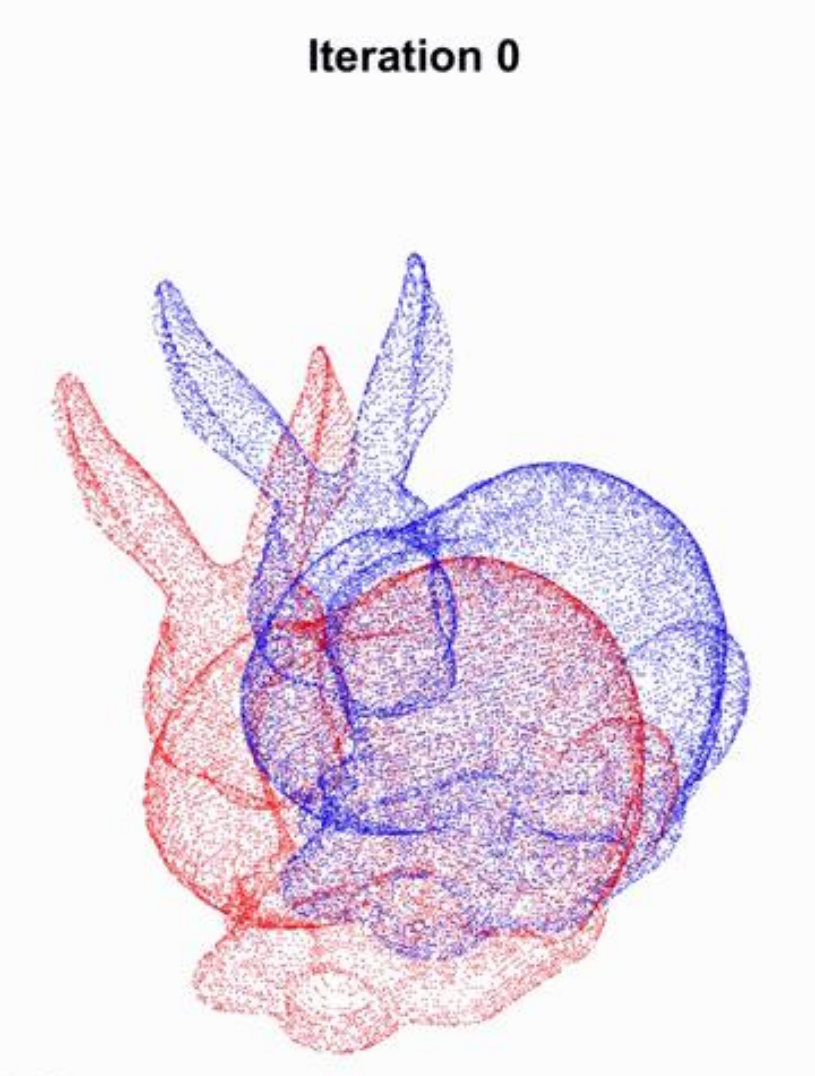
Uses point-to-point metric



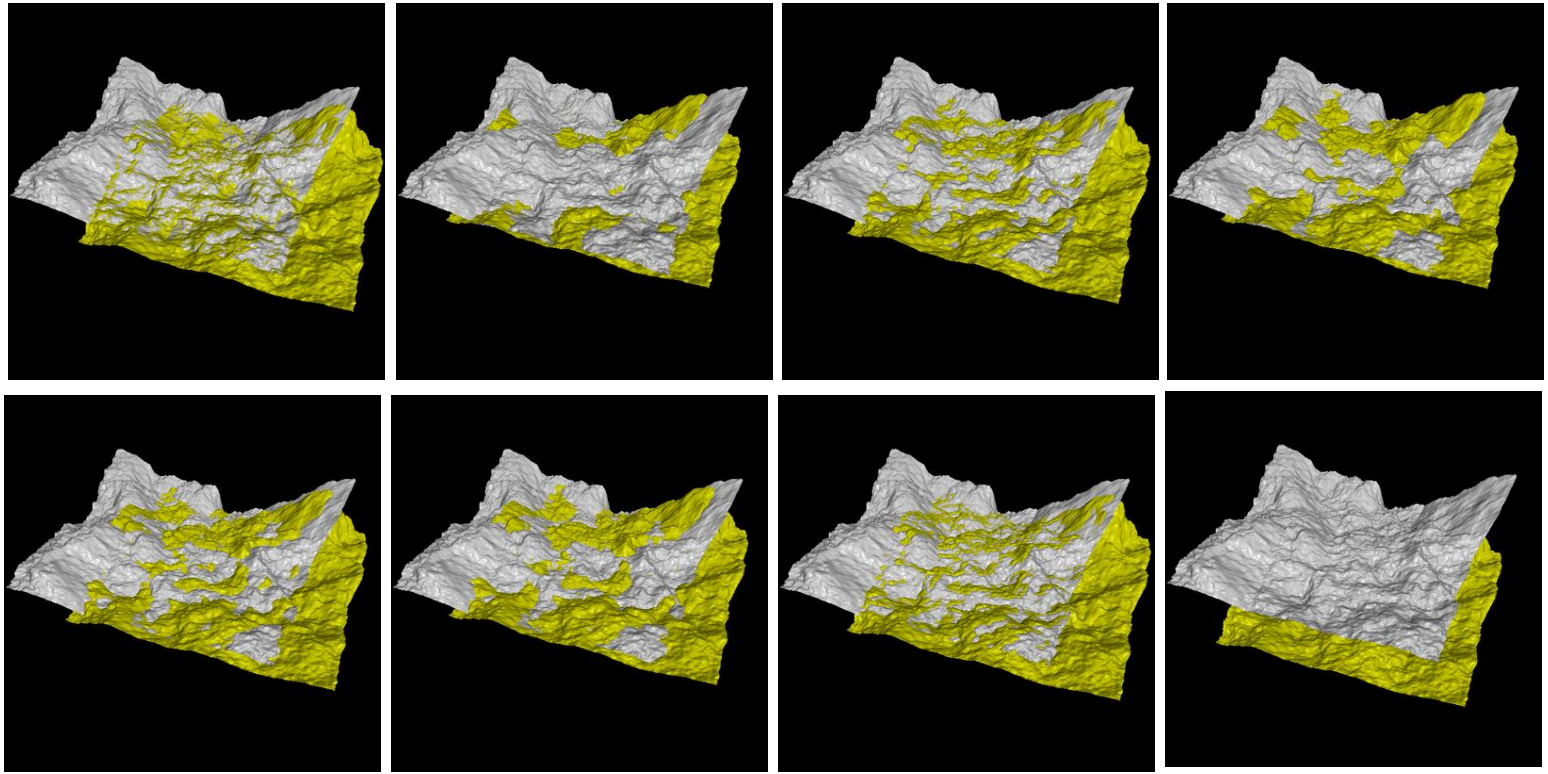
Repeats iterations of algorithm until convergence to solution

ICP Example

- Two point clouds of a 3D model of a rabbit are aligned using ICP
- Vanilla ICP can be slow and the Initial guess is important for ICP to converge



Find Local Transformation to Align Points or Surfaces



This section is courtesy of Cyrill Stachniss

The Basic Alignment Problem with known correspondences

- Given two point sets (point clouds):

  $Y = \{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_I\}$ $\qquad$ $X = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_J\}$

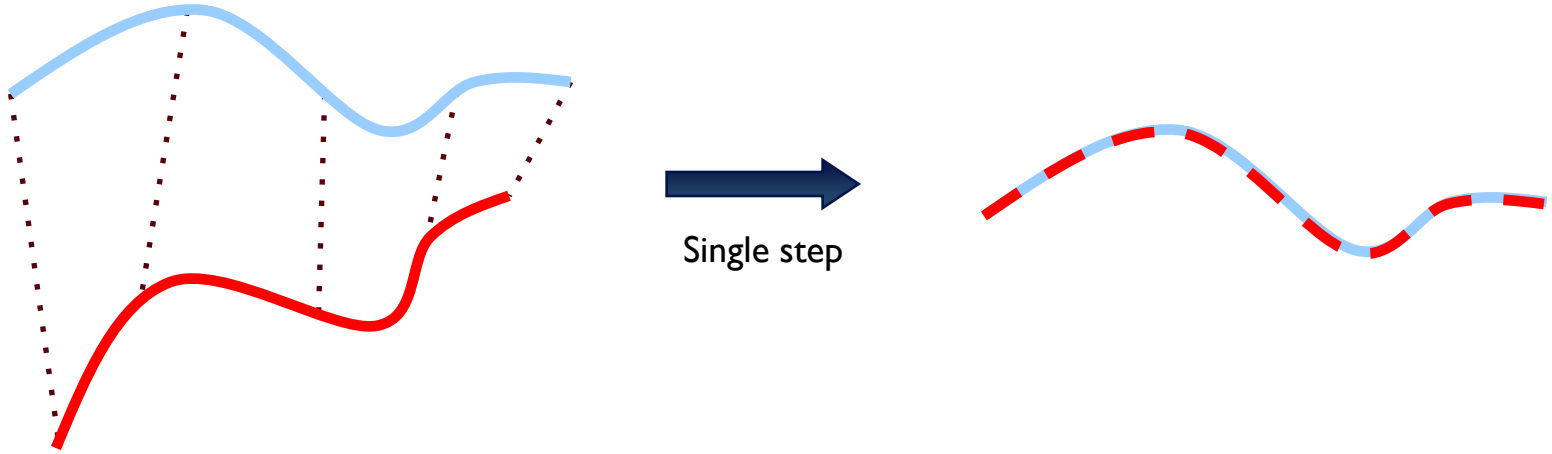
  with correspondences $C = \{(i, j)\}$

- Wanted: Translation $\boldsymbol{t}$ and rotation R that minimize the sum of the squared errors:

$$\sum_{(i,j)\in C} \|\boldsymbol{y}_i - (R\boldsymbol{x}_j - \boldsymbol{t})\|^2 \rightarrow \min$$

Key Idea

If the correct correspondences are known, the correct relative rotation/translation can be computed directly



Monte Carlo Localization

Issues

- A mechanism to compensate the mistakes committed by odometry
- A solution robust to compensate for lack of information on initial position

Issues

- A mechanism to compensate the mistakes committed by odometry
- A solution robust to compensate for lack of information on initial position

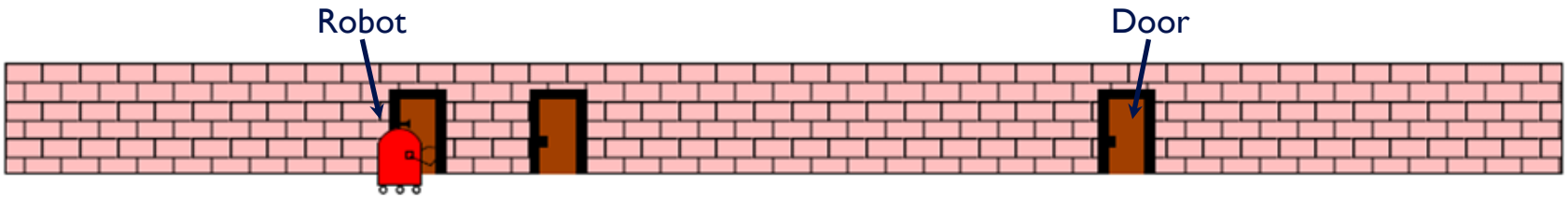
Solution: Monte Carlo Localization

(using particle filters)

Alternate Solutions: Kalman Filter, Topological Markov Localization

Particle Filter: Intuition

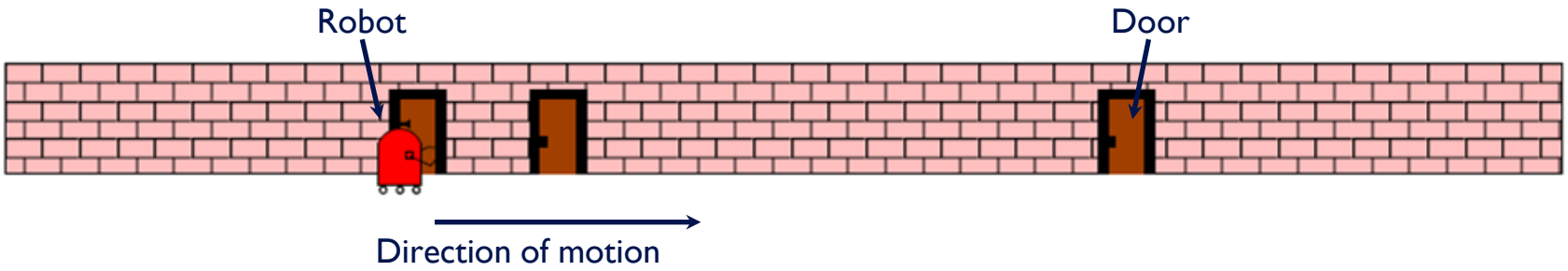
A Example in 1 Dimension



Courtesy: Thrun, Burgard, Fox

Particle Filter

A Example in 1 Dimension

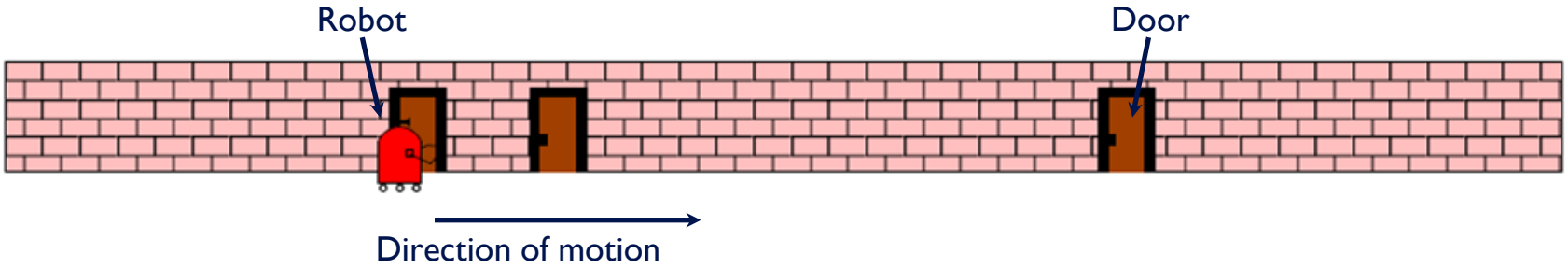


Belief State

Particle Filter

A Example in 1 Dimension

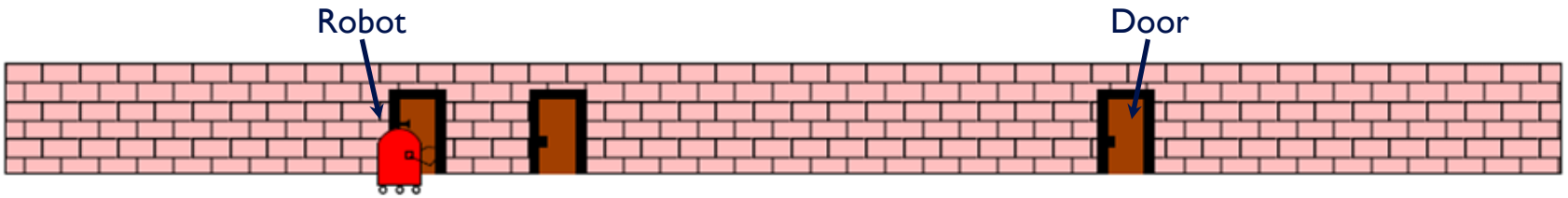
At time t = 1



Particle Filter

A Example in 1 Dimension

At time t = 1

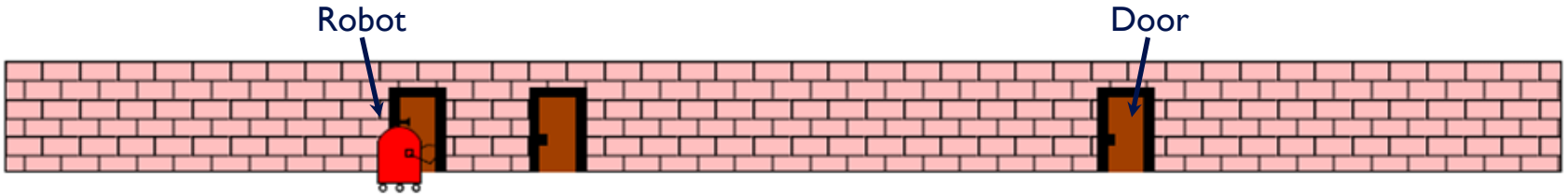


Measurement Model

Particle Filter

A Example in 1 Dimension

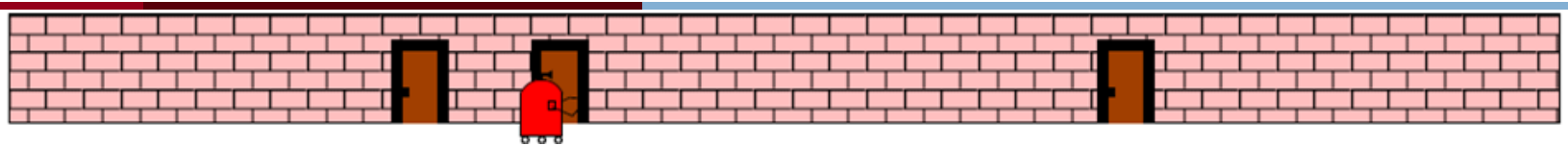
At time t = 1



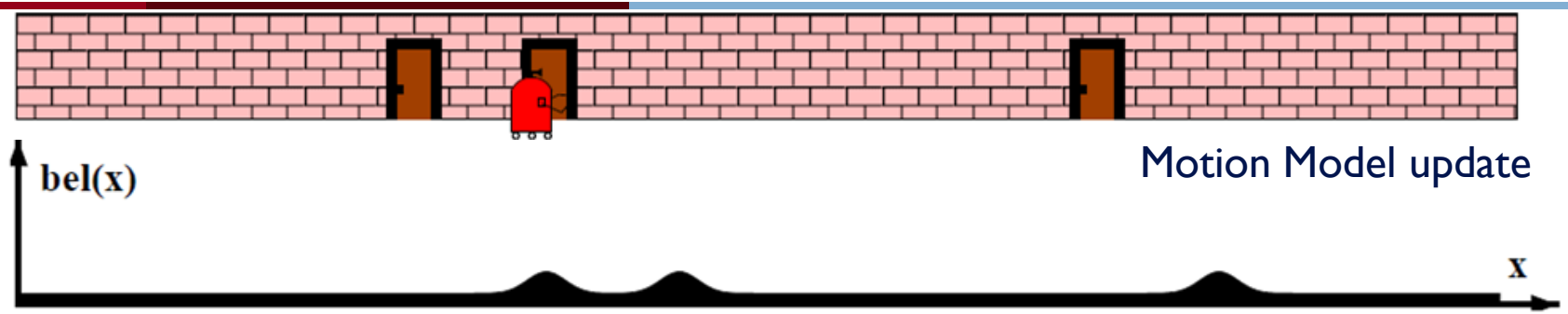
Measurement Model

Belief State

At time t = 2, robot moves forward a certain distance



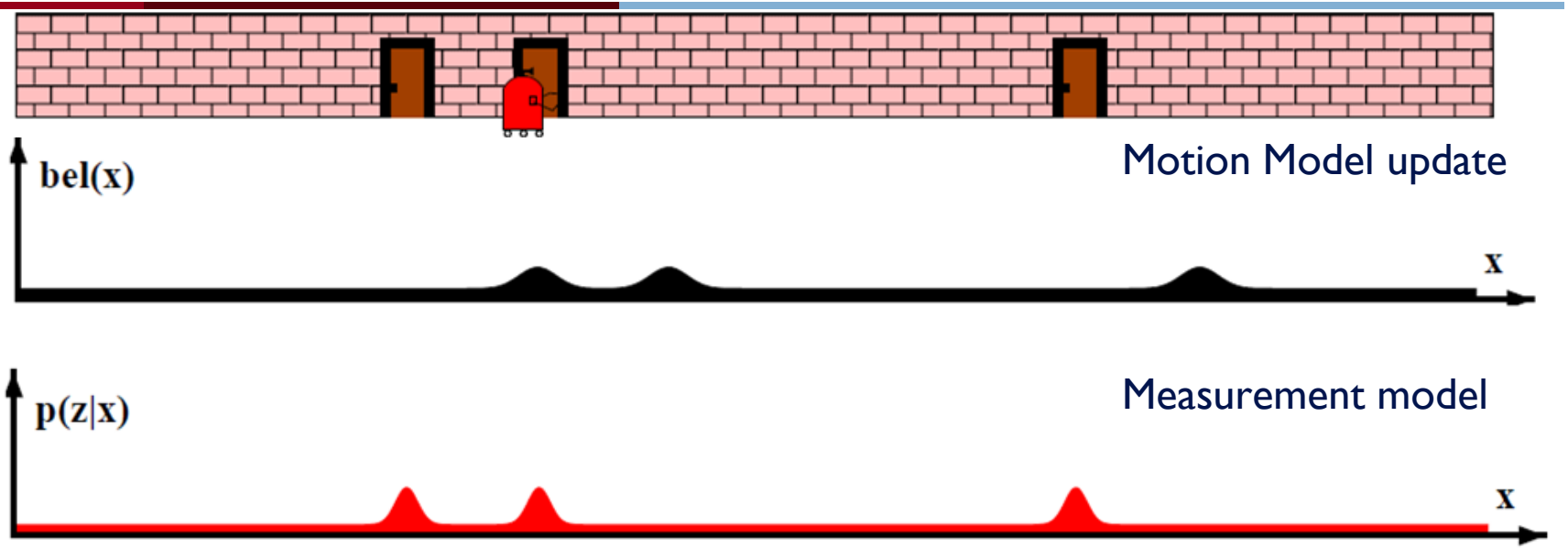
At time t = 2, robot moves forward a certain distance



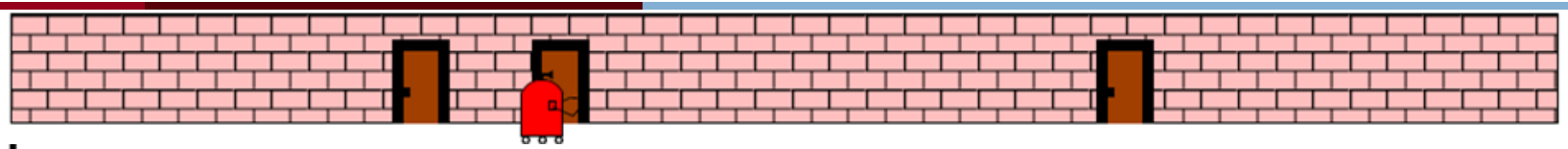
belief state shifted forward by the same distance
with some added noise to account for the error

We can never be certain where the robot moved.
So these things are a little bit flatter

At time t = 2, robot moves forward a certain distance



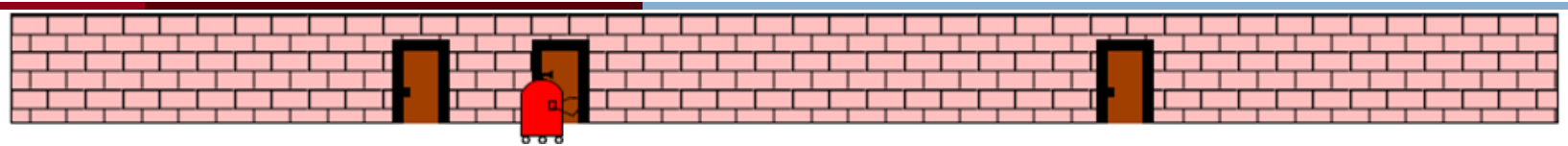
At time t = 2, robot moves forward a certain distance



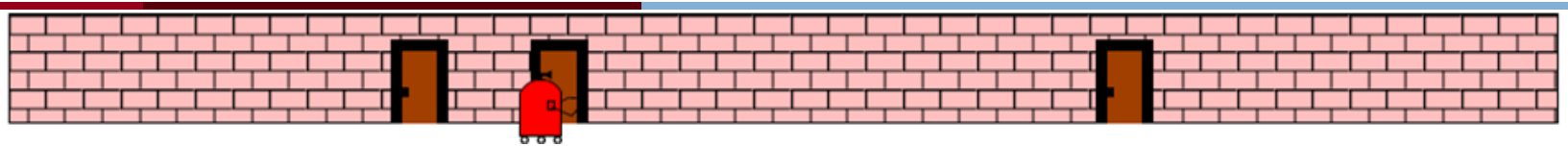
Motion Model update

Measurement model

Belief State



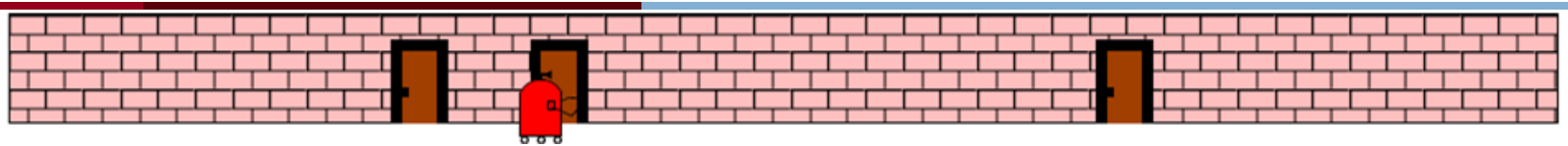
Practically the belief state is not used as a continuous function which makes it computationally expensive.



Continuous State

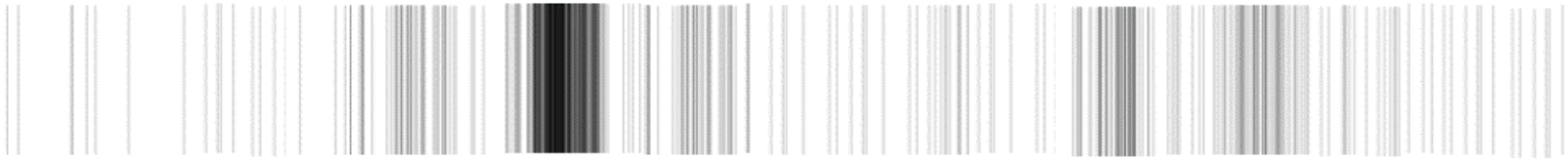
Each of these sample position is called a particle

Discrete State



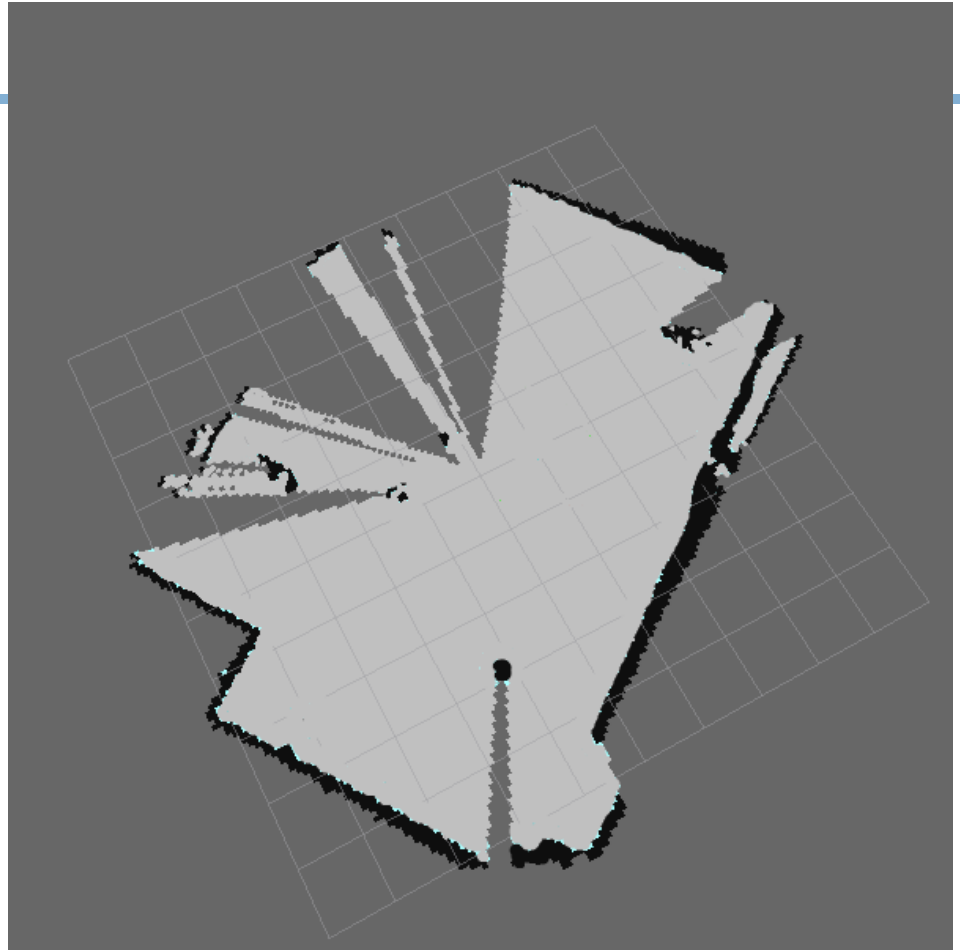
Continuous State

Discrete State



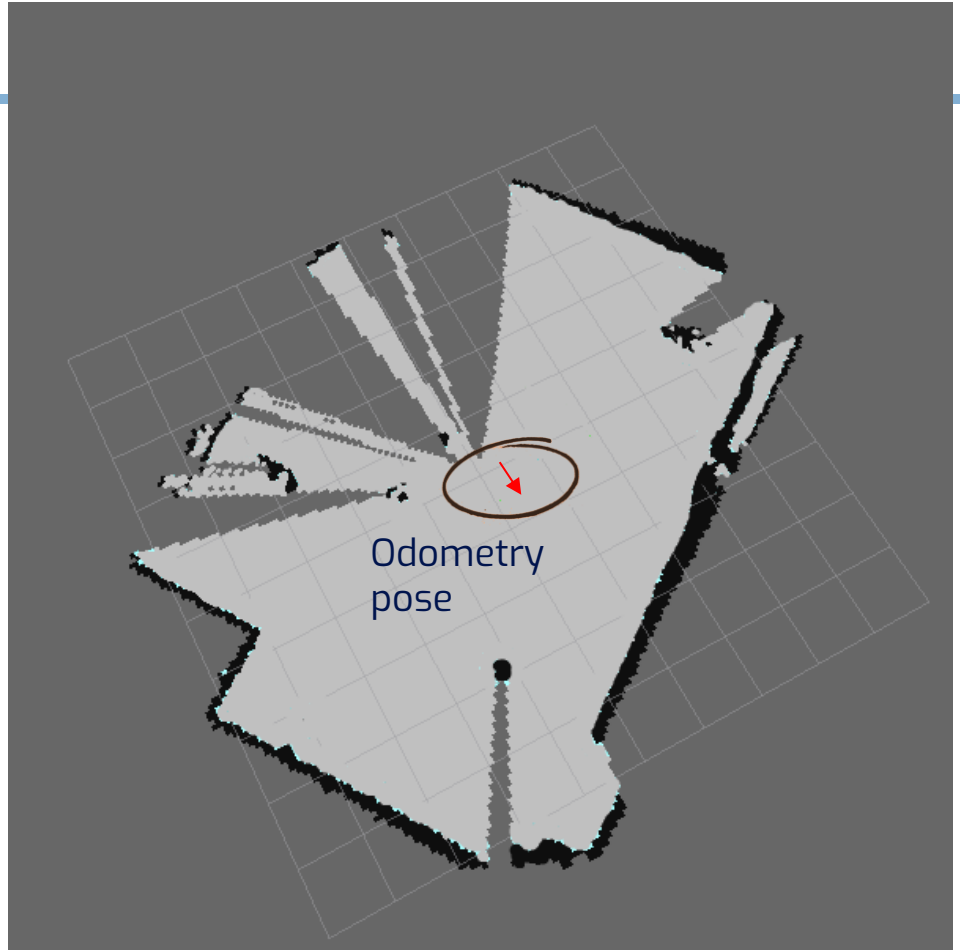
Particle Filter in 2D

- This is a map previously generated using hector mapping.
- The black pixels on the map denote the walls and the grey pixels denote free space.



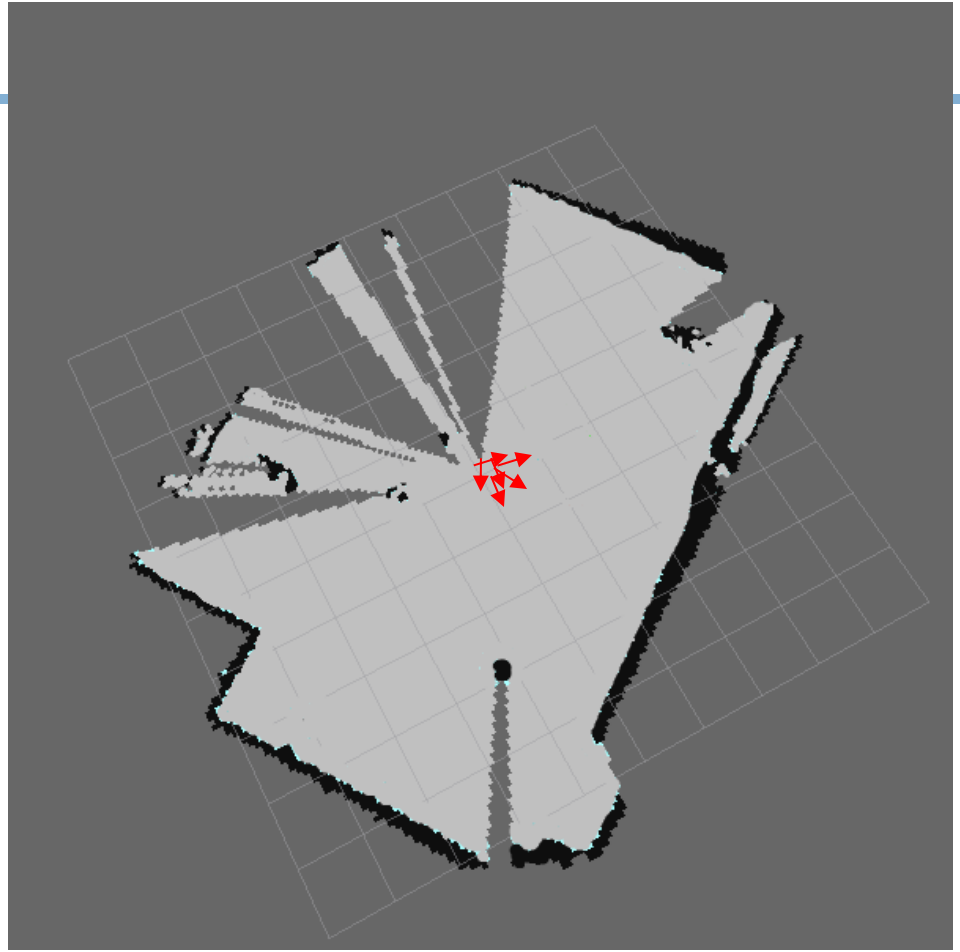
Particle Filter in 2D

- The red arrow indicates our pose obtained from odometry which is not accurate.
- Let's use particle filters to minimize the error and localize more accurately in the map.



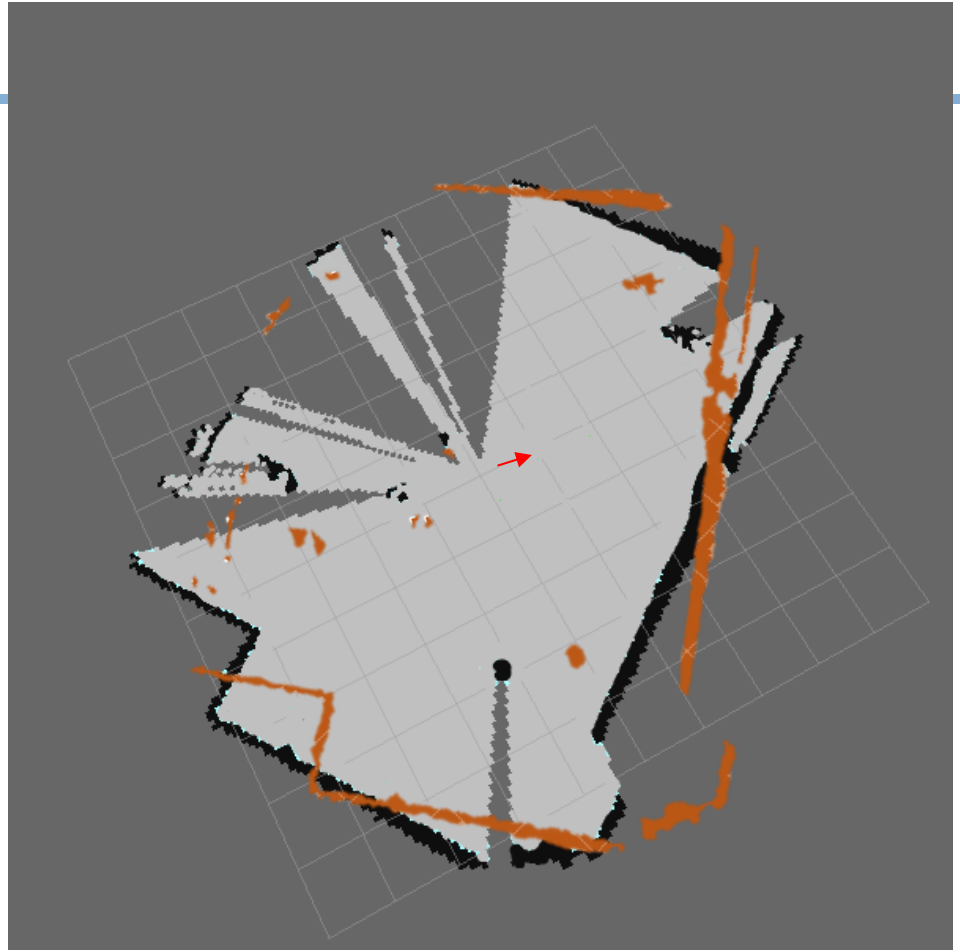
Particle Filter in 2D

- First we need to generate a set of hypothesis for our position.
- These are discrete particles drawn from a Gaussian distribution of mean being our odometry pose
- We end up with a set of poses which are obtained by adding noise to our odometry pose.



Scan Correlation

- For each particle, we orient the laser scan shown in orange with respect to the particle's pose and compute a correlation score

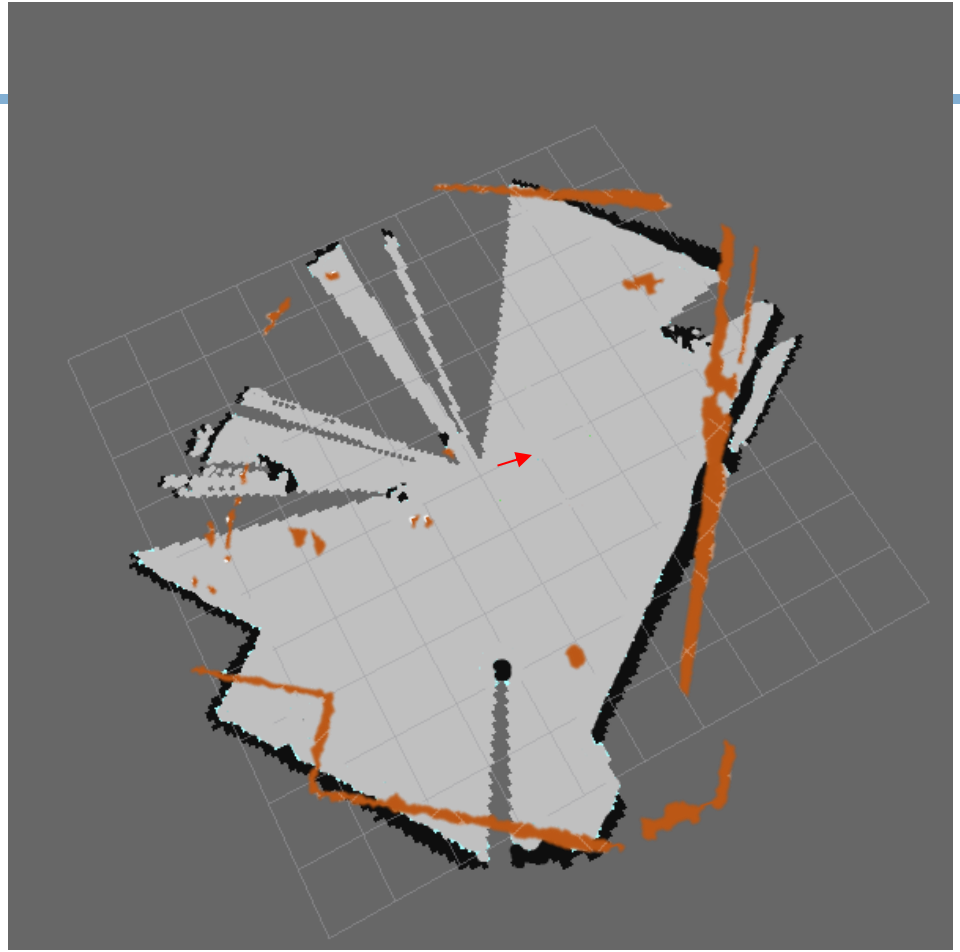


Scan Correlation

map mean scan

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

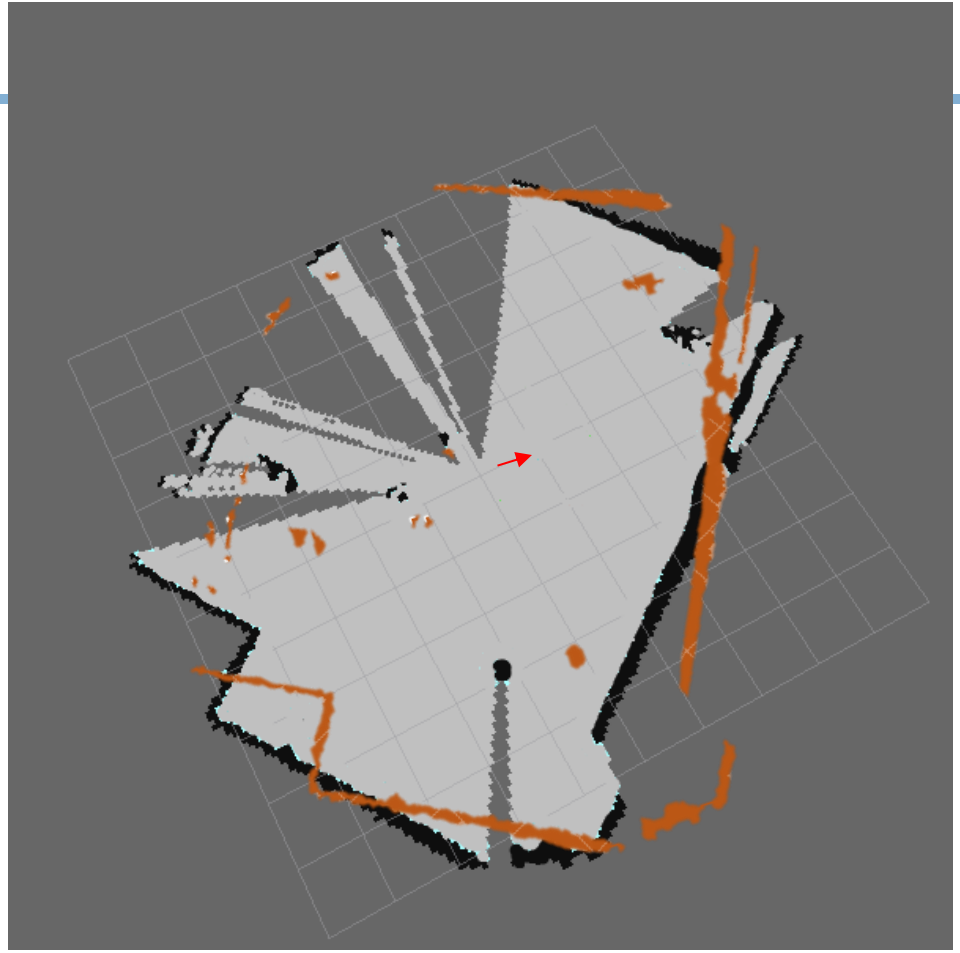
- Walls have a value of 1 and free spaces have value of 0. *m* and *n* are the *x* and *y* coordinates of the pixels.
- This equation basically counts the number of black and orange pixels (i.e our wall pixels) that overlap.



Scan Correlation

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

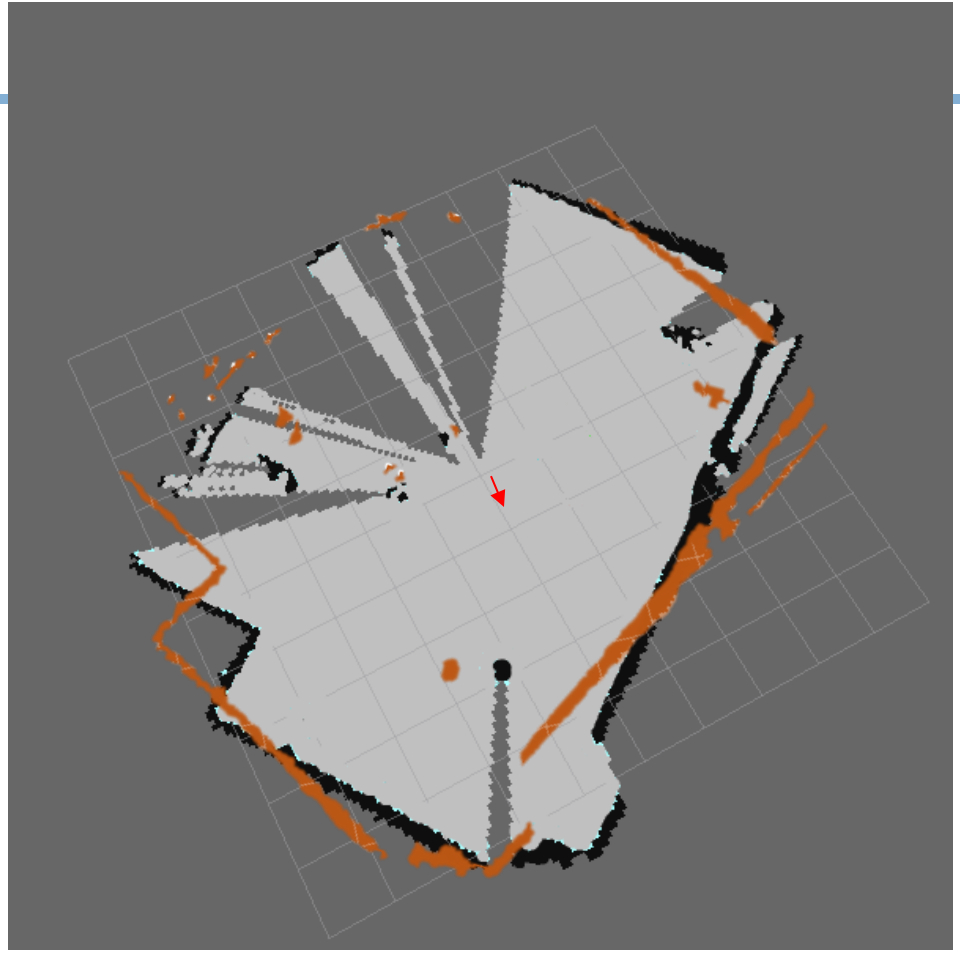
Particle	Weight
Particle 1	S_1



Scan Correlation

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

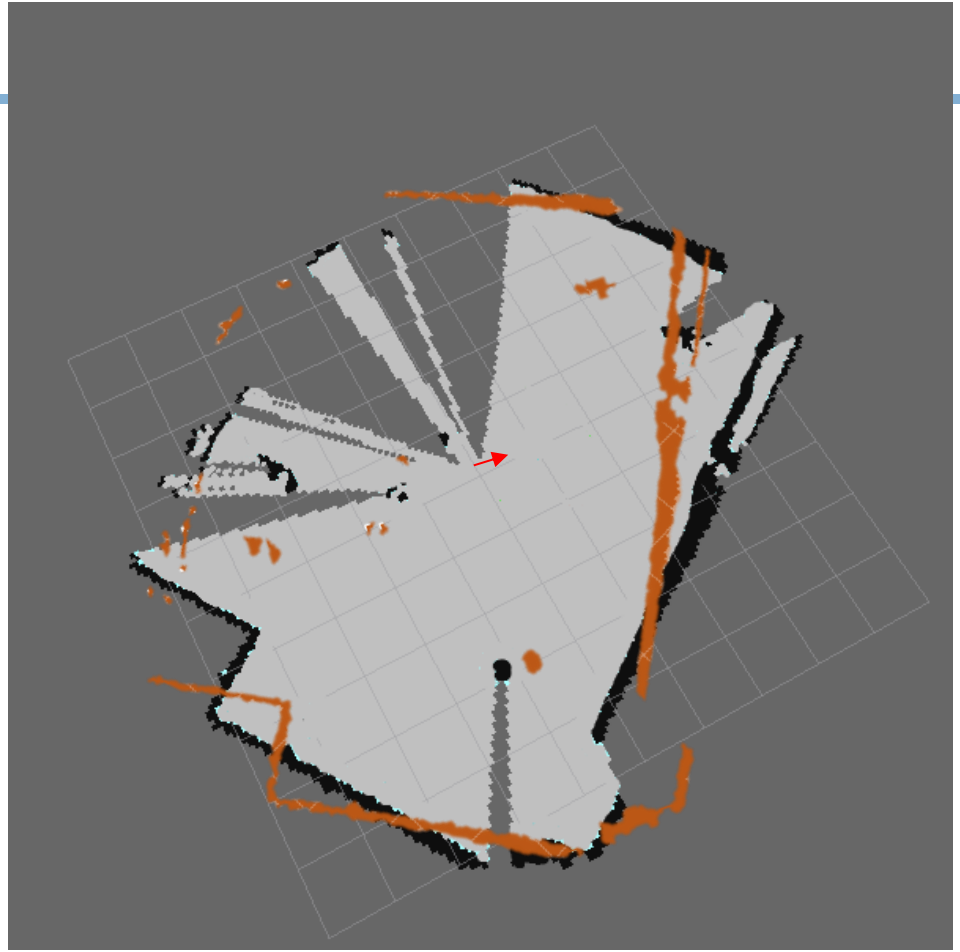
Particle	Weight
Particle 1	S_1
Particle 2	S_2



Scan Correlation

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

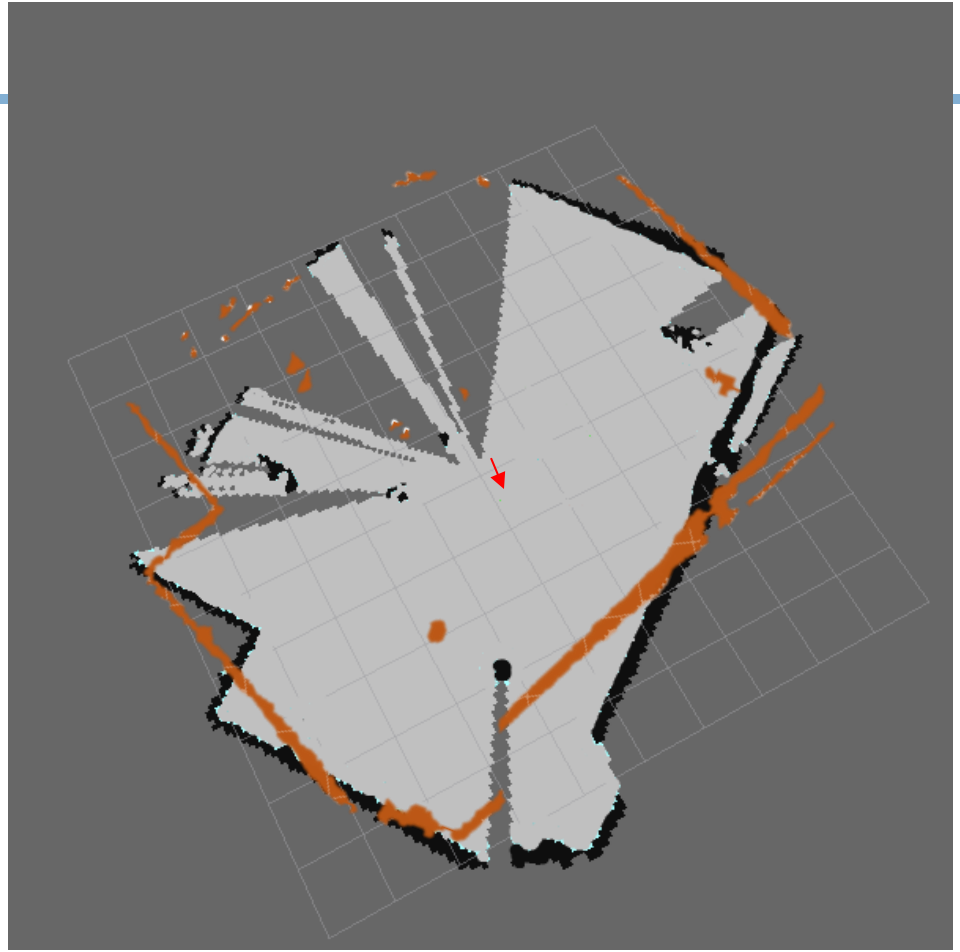
Particle	Weight
Particle 1	S_1
Particle 2	S_2
Particle 3	S_3



Scan Correlation

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

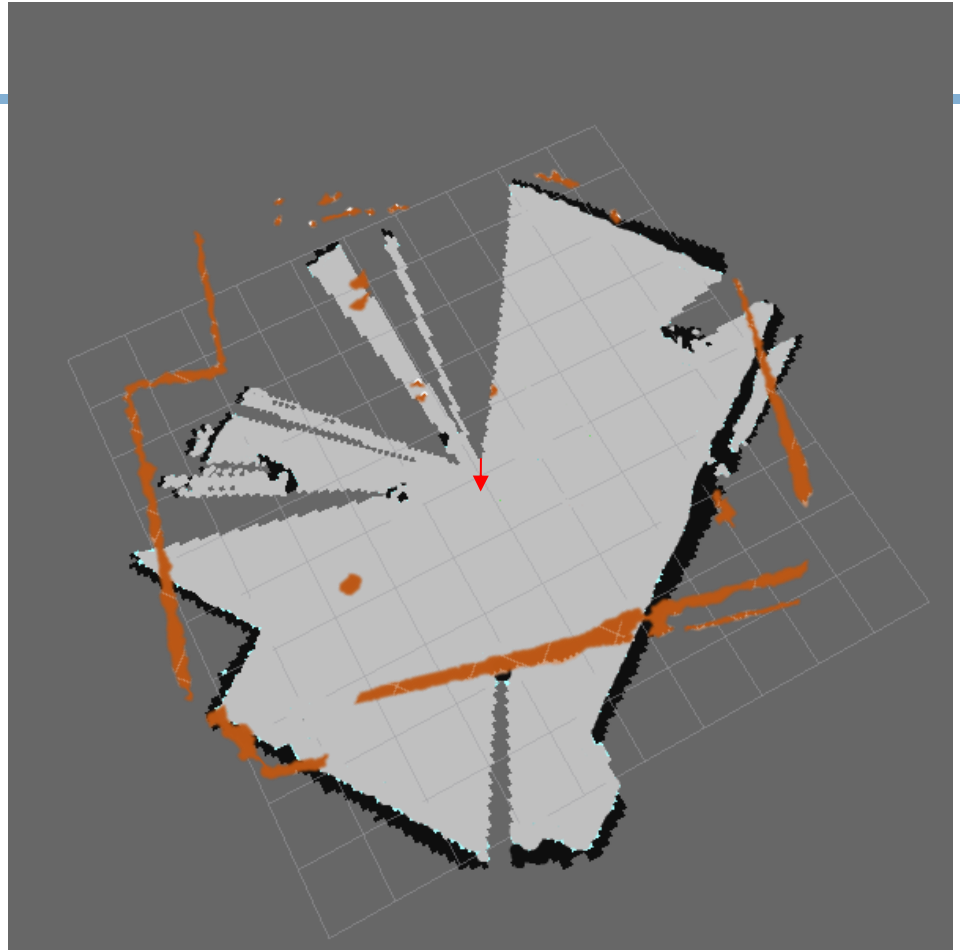
Particle	Weight
Particle 1	S_1
Particle 2	S_2
Particle 3	S_3
Particle 4	S_4



Scan Correlation

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

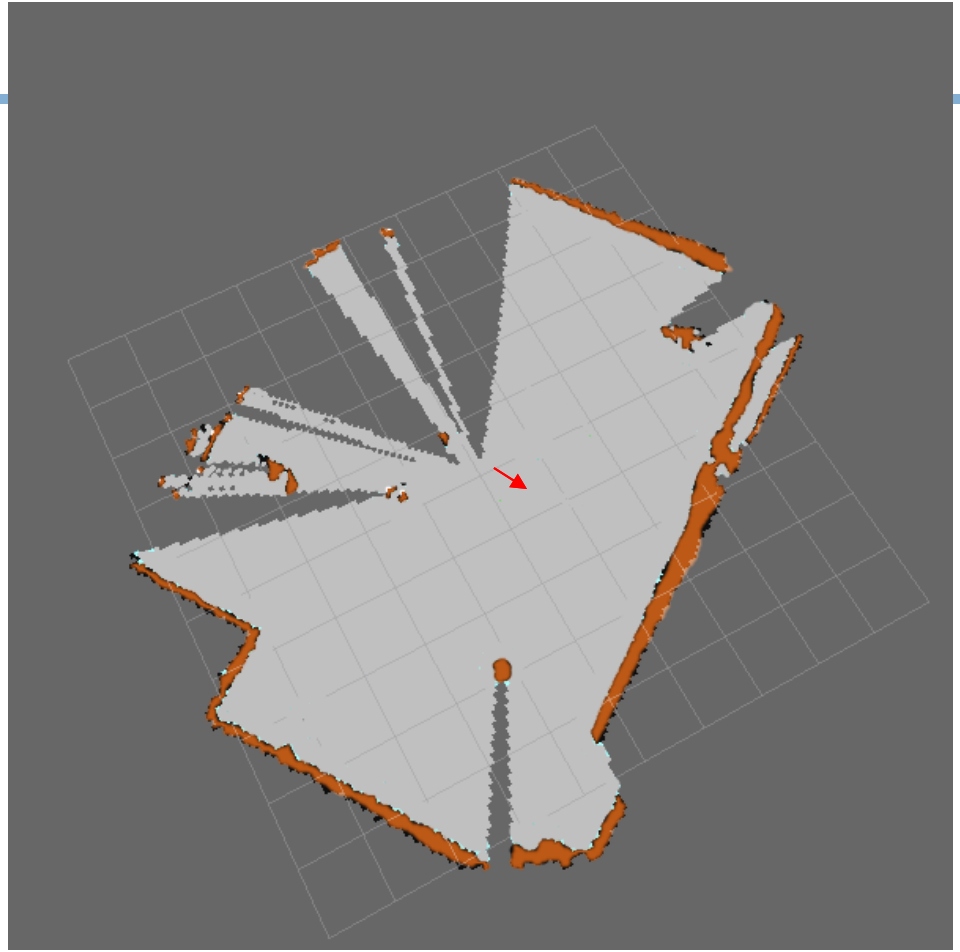
Particle	Weight
Particle 1	S_1
Particle 2	S_2
Particle 3	S_3
Particle 4	S_4
Particle 5	S_5



Scan Correlation

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

Particle	Weight
Particle 1	S_1
Particle 2	S_2
Particle 3	S_3
Particle 4	S_4
Particle 5	S_5
Particle 6	S_6

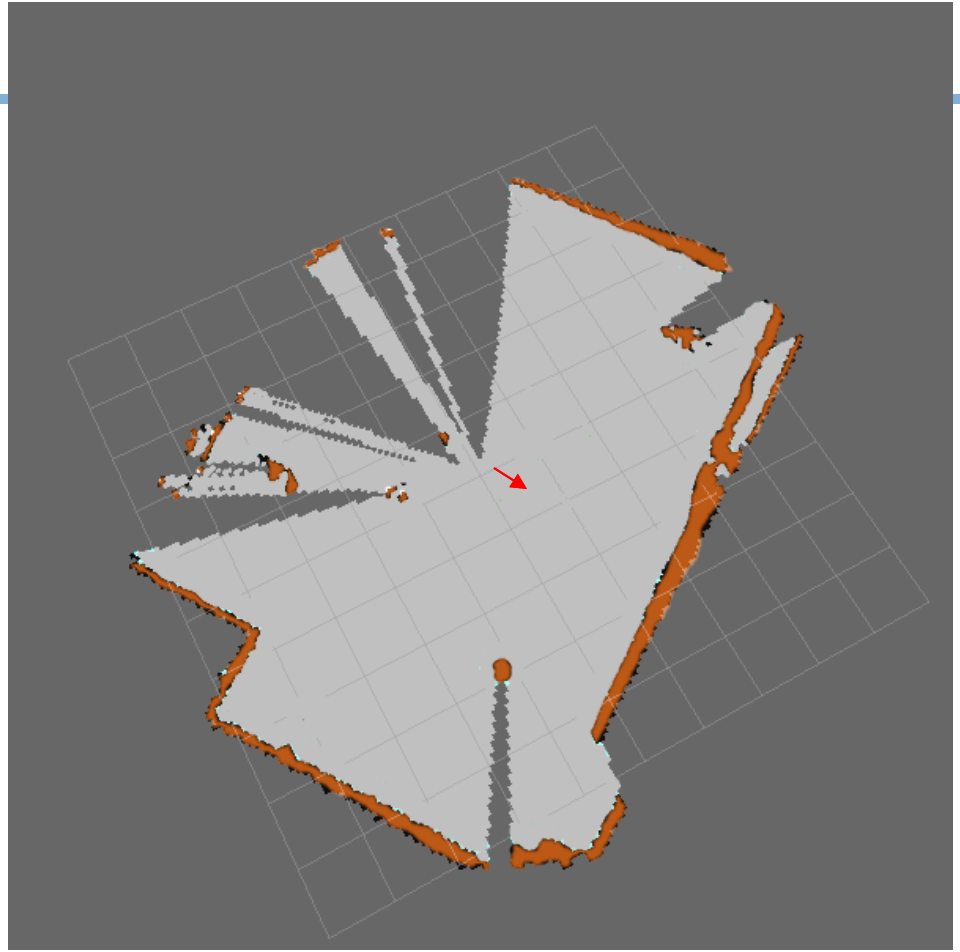


Scan Correlation

$$S = \frac{\sum_m \sum_n (A_{mn} - \overline{A})(B_{mn} - \overline{B})}{\sqrt{\left(\sum_m \sum_n (A_{mn} - \overline{A})^2\right)\left(\sum_m \sum_n (B_{mn} - \overline{B})^2\right)}}$$

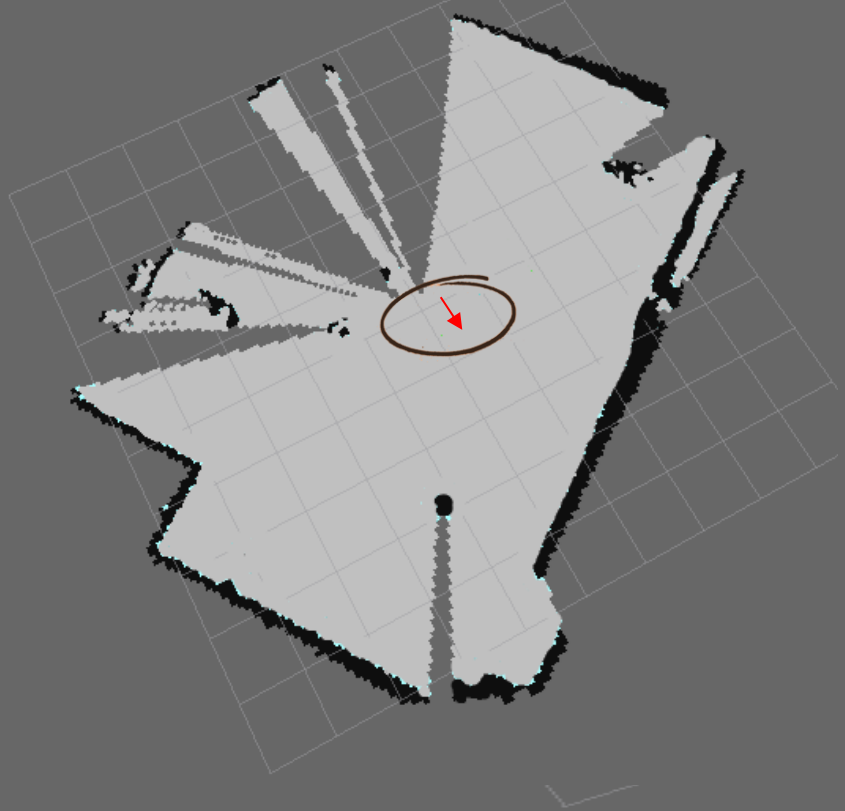
Particle	Weight
Particle 1	S_1
Particle 2	S_2
Particle 3	S_3
Particle 4	S_4
Particle 5	S_5
Particle 6	**S_6**

Particle 6 with the maximum weight is chosen as the pose at the current state.

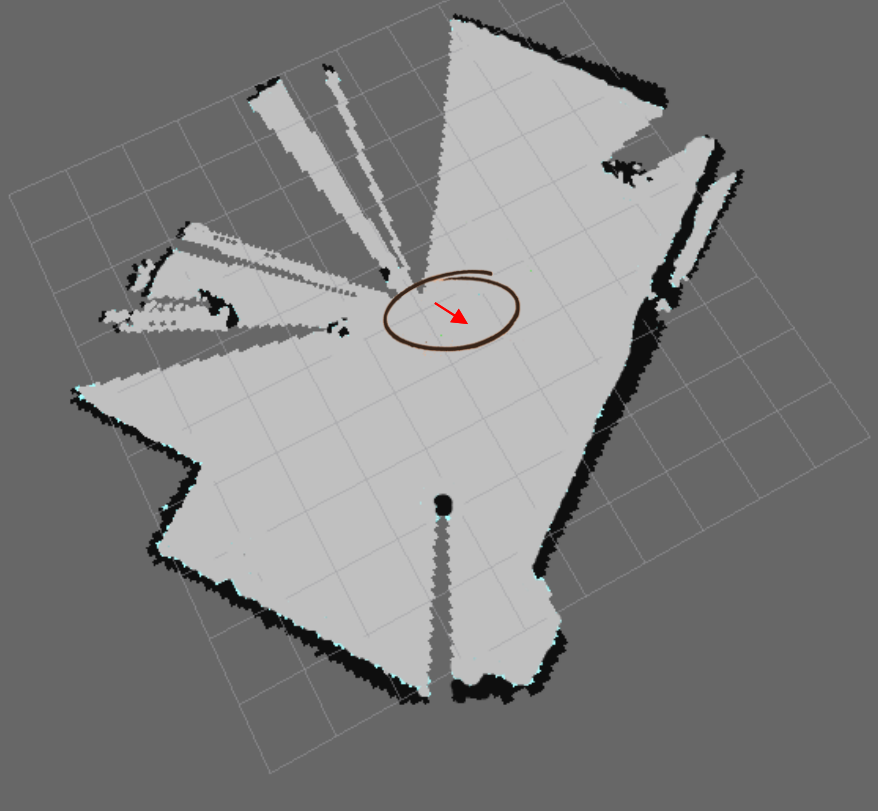


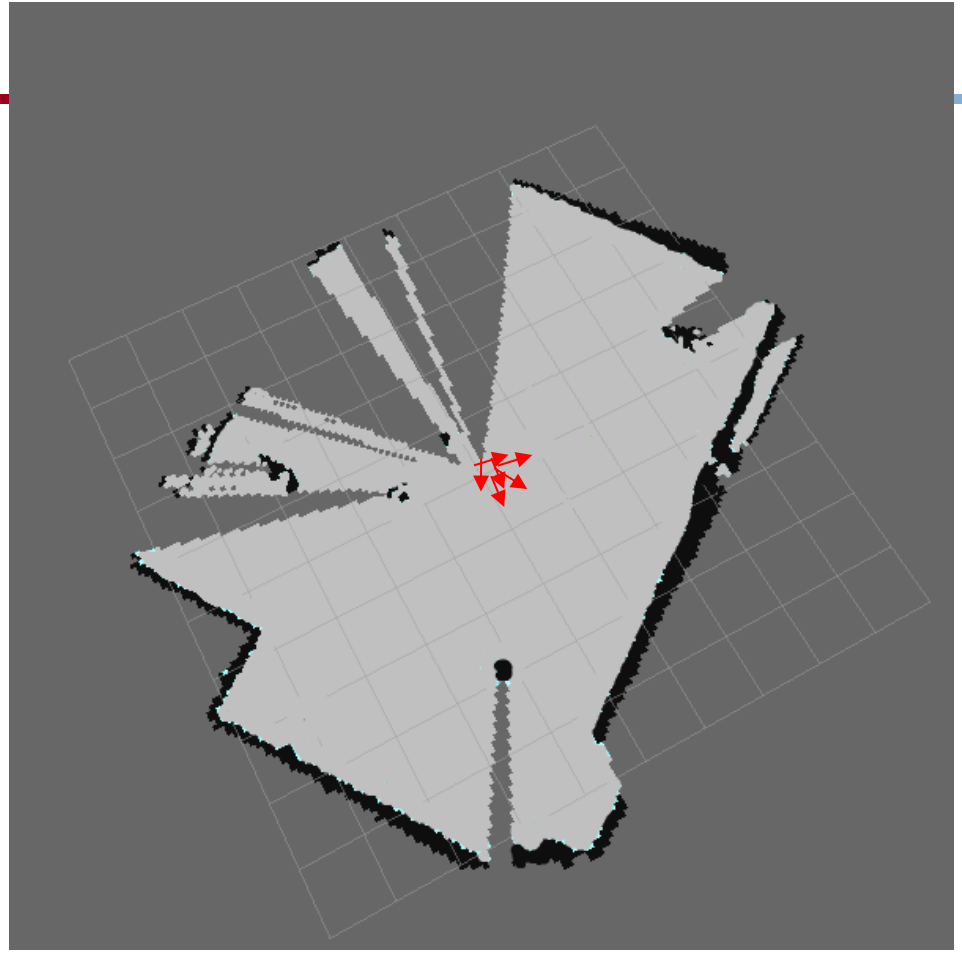
Localization using

Odometry



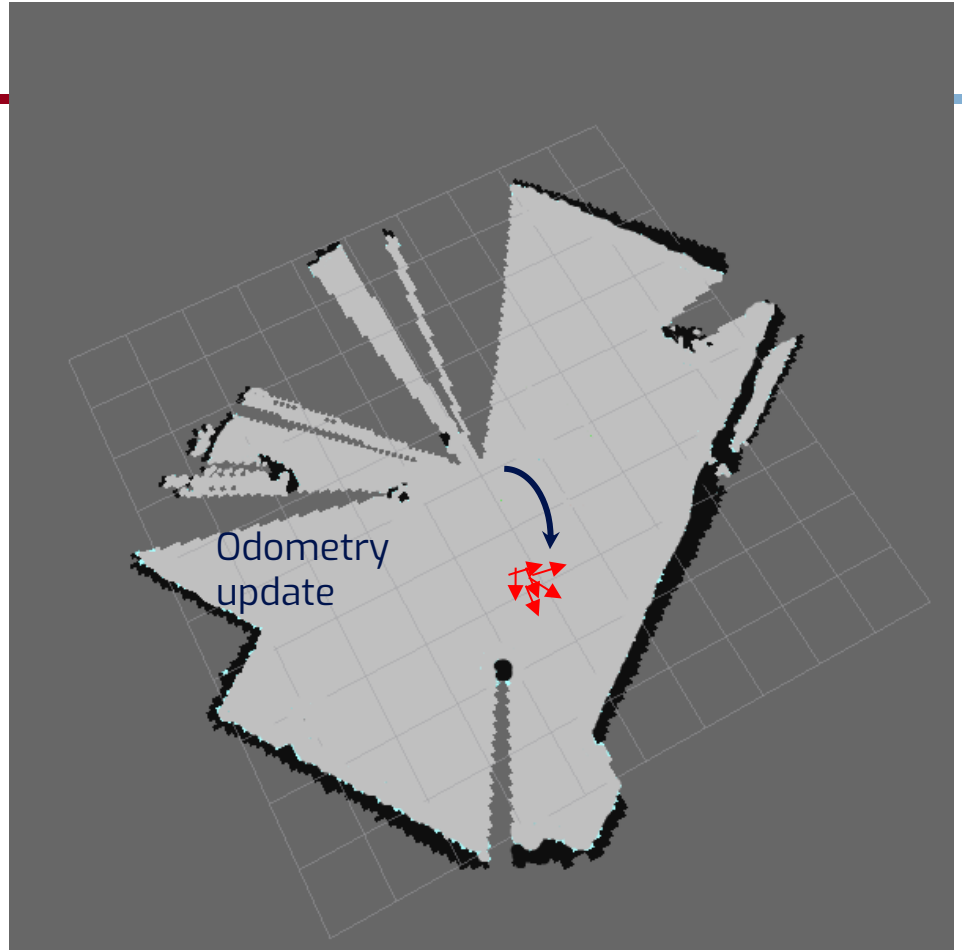
Particle Filter





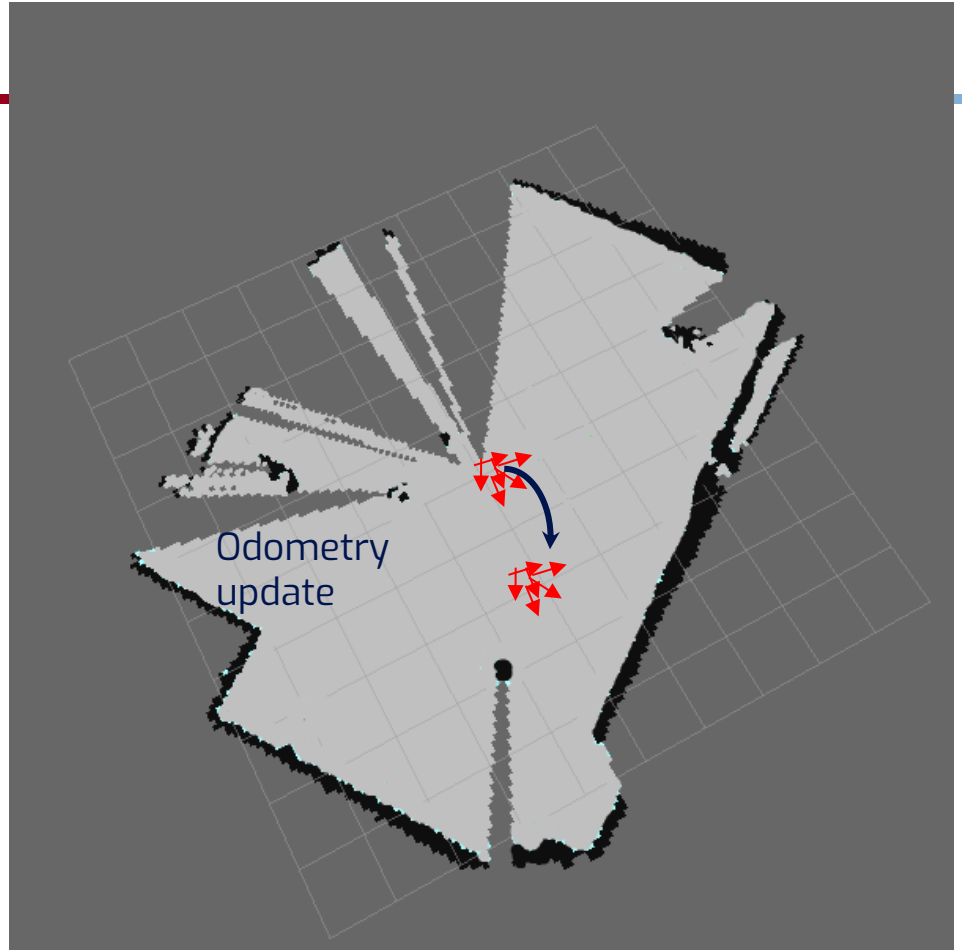
Update Step

- Update the particle cloud with the update in position from the odometry
- Repeat Scan matching process for each particle and determine the weights.



Update Step

- Update the particle cloud with the update in position from the odometry
- Repeat Scan matching process for each particle and determine the weights.



Update Step

- Update the particle cloud with the update in position from the odometry
- Repeat Scan matching process for each particle and determine the weights.

The scores are first normalized and then multiplied with the previous weight for that particle at time *t*-1.

Particle Weights

$$W_t \leftarrow W_{t-1} \times S$$

Particle Filters: Analysis

Particle Filters: Problem Definition

- Estimating the state of a dynamical system is fundamental problem
- The recursive Bayes Filter is an effective approach to estimate the belief about the state of a dynamical system
  - How to represent this belief?
  - How to maximize it?

Particle Filters: Problem Definition

- Estimating the state of a dynamical system is fundamental problem
- The recursive Bayes Filter is an effective approach to estimate the belief about the state of a dynamical system
  - How to represent this belief?
  - How to maximize it?
- Particle filters are a way to **efficiently** represent an arbitrary **(non-Gaussian) distribution**
- Basic Principle
  - Set of state hypothesis ('particles')
  - Survival of the fittest

Bayes Recursive Filter

Recall that we have:

Integrate out over previous beliefs. In practice finite approximation

$$Bel(x_t) = \eta \; p(o_t \mid x_t) \int p(x_t \mid x_{t-1}, a_{t-1}) Bel(x_{t-1}) dx_{t-1}$$

Normalization Constant
Make sure everything adds up to 1!

Observation Likelihood
Sensor Model
Compute how likely your measurements were given updated particles.

Transition Model
Motion Model
Simulate noisy dynamics of particles based on control input.

Previous Belief
Draw your particles with replacement according to importance weight.

Read right to left

In practice we represent the distributions non-parametrically using particles (a finite set of samples)

Gaussian Filters

The Kalman filter and its variants can only model **Gaussian distributions**

$$p(x) = \det(2\pi\Sigma)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu) \right)$$

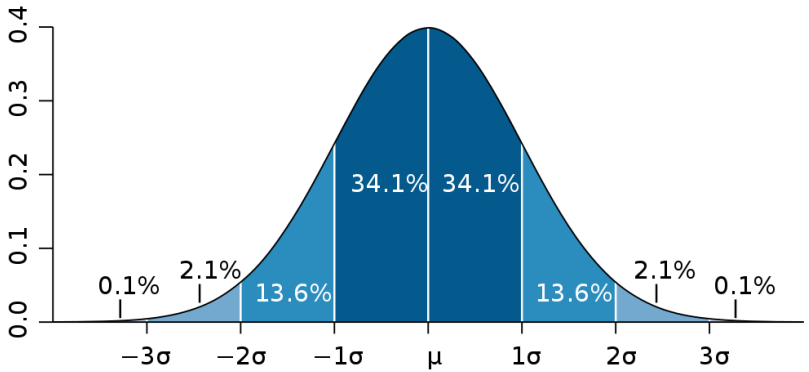
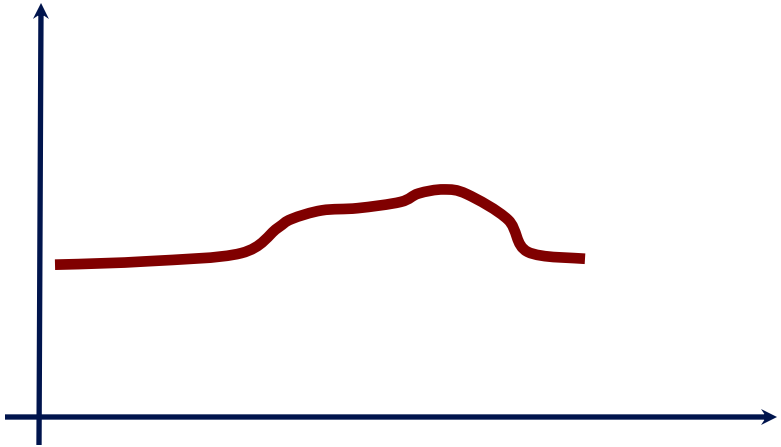


Image courtesy: K. Arras

Flexible Function Approximation

Goal: approach for dealing with **arbitrary distributions**

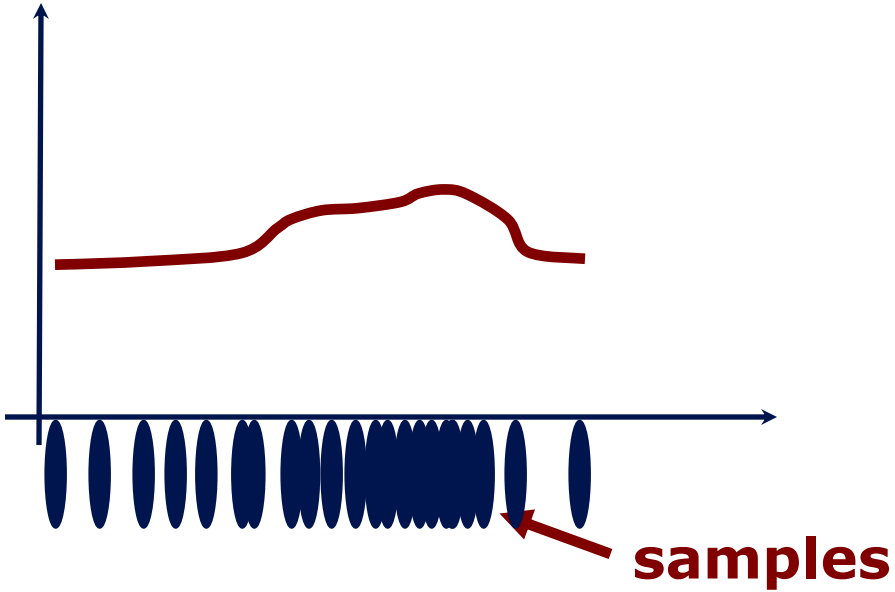
The key idea of the particle filter is not to use a parametric form like Gaussian but to use a **non-parametric representation** of the distribution



Key Idea: Samples

Multiple samples to represent arbitrary distributions

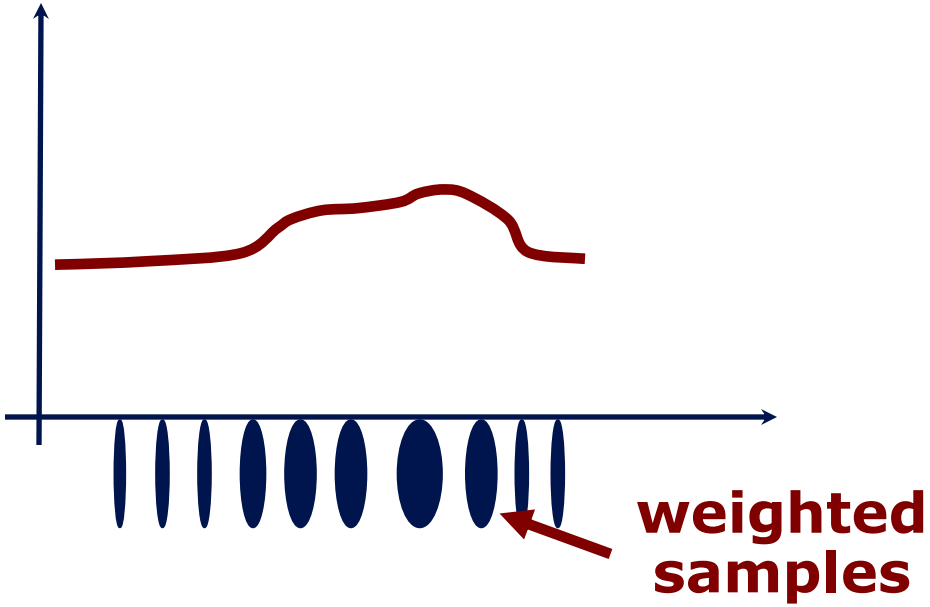
The particle filter uses random samples



Key Idea: Weighted Samples

Multiple **weighted samples** to represent arbitrary distributions

We may also assign a weight to those samples so bigger the oval here, the higher the weight and the higher the probability of the corresponding area



Particle Set

- Set of weighted samples

$$\mathcal{X} = \left\{ \left\langle x^{[j]}, w^{[j]} \right\rangle \right\}_{j=1,\ldots,J}$$

state hypothesis

importance weight

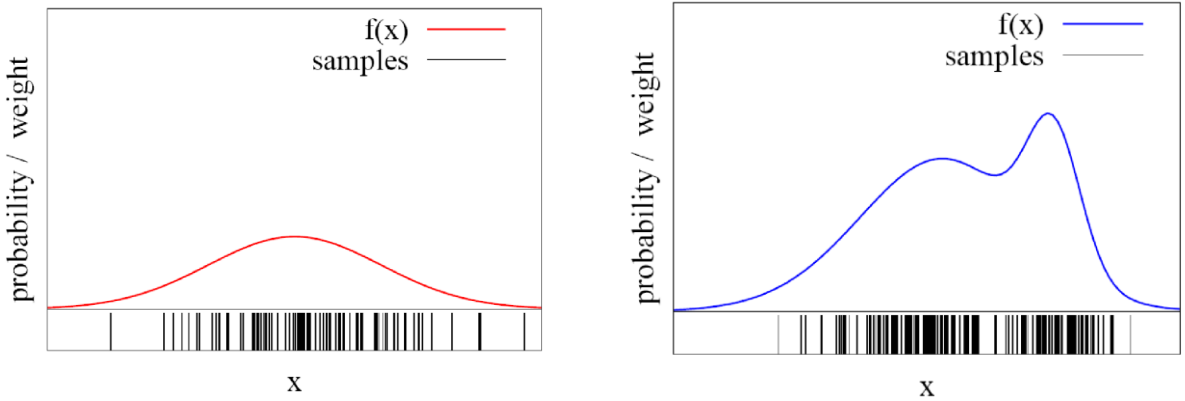
- The samples represent the posterior

$$p(x) = \sum_{j=1}^{J} w^{[j]} \, \delta_{x^{[j]}}(x)$$

Dirac delta function

Particles for Approximation

- Particles for function approximation

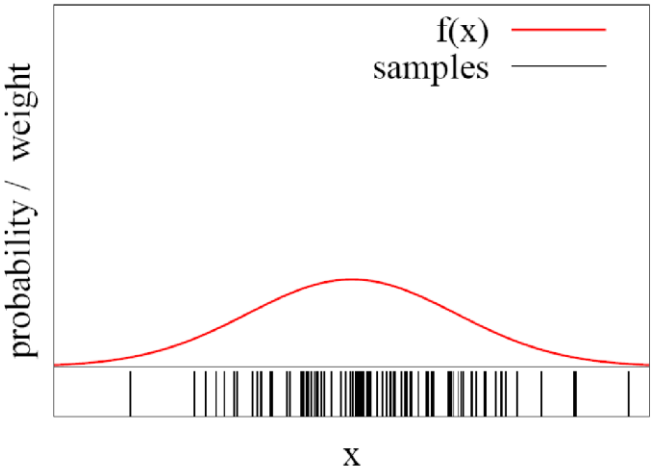


- The more particles fall into a region, the higher the probability of the region

How to obtain such samples?

Closed-form sampling is only possible for a few distributions

- Example: Gaussian



We can use this closed form to just take 12 random numbers between minus and plus the standard deviation of the Gaussian distribution, sum all of them up and divided by 2 (variance equals to 1, so -1 to 1 is a range of 2)

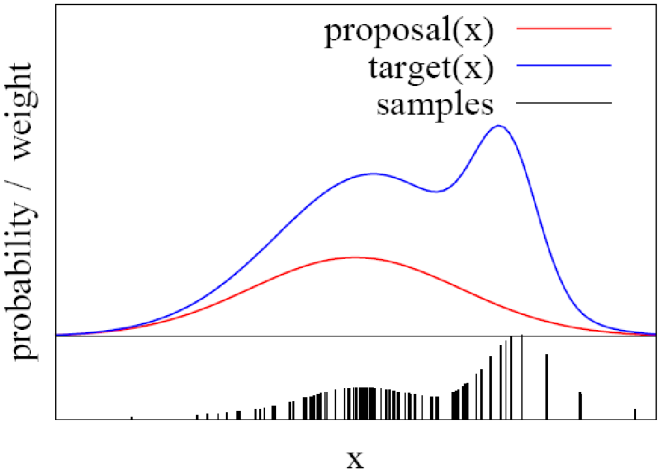
$$x \leftarrow \frac{1}{2}\sum_{i=1}^{12} \text{rand}(-\sigma, \sigma)$$

How to sample from **other** distributions that cannot be represented by an equation?

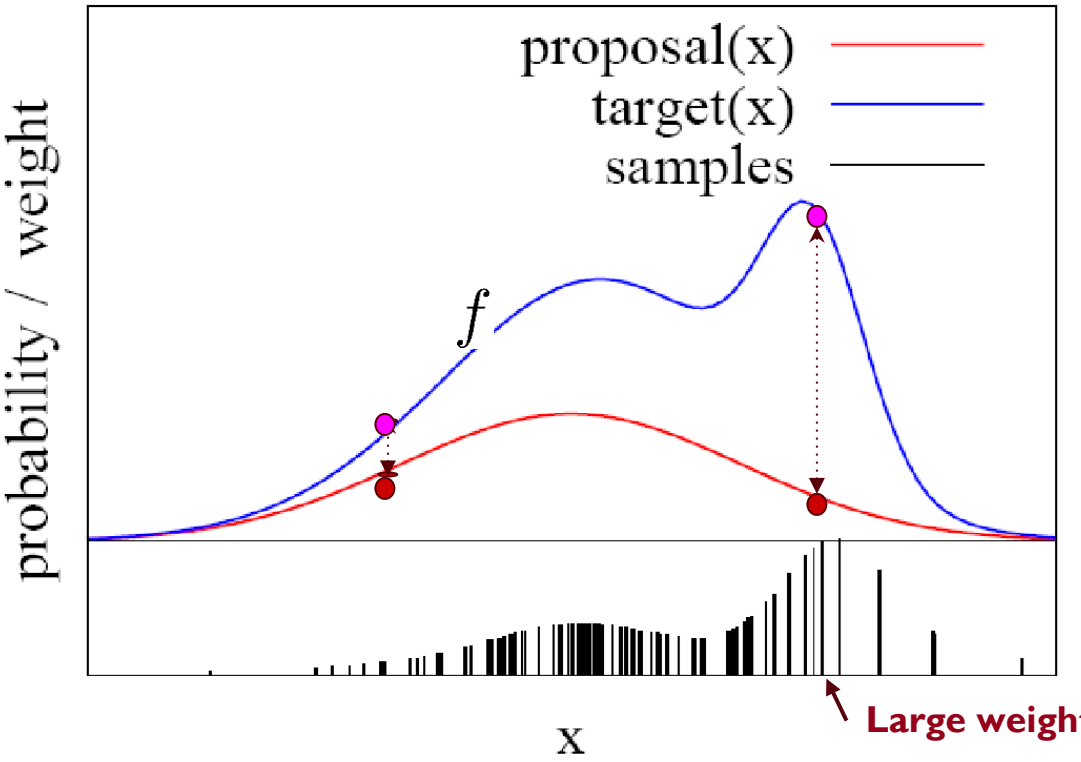
Importance Sampling Principle

- We can use a different distribution π to generate samples from f
- Account for the "differences between π and f" using an importance weight $\omega = f(x)/\pi(x)$
- Target f
- Proposal π
- Pre-condition:

$$f(x) > 0 \rightarrow \pi(x) > 0$$



Importance Sampling Principle



$$\omega = f(x)/\pi(x)$$

Particle Filter for Dynamic State Estimation Problems

- Recursive Bayes filter (uses the importance sampling principle to update the belief)
- Non-parametric approach
- Models the distribution by samples
- **Prediction:** draw from the proposal
- **Correction:** weighting by the ratio of the target and proposal

The more samples we use,
the better is the estimate!

Particle Filter Algorithm

1. Sample the particles using the proposal distribution

$$x_t^{[j]} \sim proposal(x_t \mid \ldots)$$

2. Compute the importance weights

$$w_t^{[j]} = \frac{target(x_t^{[j]})}{proposal(x_t^{[j]})}$$

3. Resampling: Draw sample i with probability $w_t^{[i]}$ and repeat J times

Particle Filter Algorithm

Particle_filter$(\mathcal{X}_{t-1}, u_t, z_t)$: ← Input: Old sample set, controls, observations

1: $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$ ← Predicted set and Corrected set

2: *for* $j = 1$ *to* J *do*

3: *sample* $x_t^{[j]} \sim \pi(x_t)$ **Prediction Step**

4: $w_t^{[j]} = \frac{p(x_t^{[j]})}{\pi(x_t^{[j]})}$

5: $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[j]}, w_t^{[j]} \rangle$

6: *endfor*

7: *for* $j = 1$ *to* J *do*

8: *draw* $i \in 1, \ldots, J$ *with probability* $\propto w_t^{[i]}$

9: *add* $x_t^{[i]}$ *to* $\mathcal{X}_t$

10: *endfor*

11: *return* $\mathcal{X}_t$

Particle Filter Algorithm

Particle_filter$(\mathcal{X}_{t-1}, u_t, z_t)$**:**

1: $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$

2: *for* $j = 1$ *to* J *do*

3: *sample* $x_t^{[j]} \sim \pi(x_t)$

4: $w_t^{[j]} = \frac{p(x_t^{[j]})}{\pi(x_t^{[j]})}$ ← **Correction Step**

5: $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[j]}, w_t^{[j]} \rangle$

6: *endfor*

7: *for* $j = 1$ *to* J *do*

8: *draw* $i \in 1, \ldots, J$ *with probability* $\propto w_t^{[i]}$

9: *add* $x_t^{[i]}$ *to* $\mathcal{X}_t$

10: *endfor*

11: *return* $\mathcal{X}_t$

Particle Filter Algorithm

Particle_filter$(\mathcal{X}_{t-1}, u_t, z_t)$**:**

1: $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$
2: *for* $j = 1$ *to* J *do*
3: *sample* $x_t^{[j]} \sim \pi(x_t)$
4: $w_t^{[j]} = \frac{p(x_t^{[j]})}{\pi(x_t^{[j]})}$
5: $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[j]}, w_t^{[j]} \rangle$
6: *endfor*
7: *for* $j = 1$ *to* J *do*
8: *draw* $i \in 1, \ldots, J$ *with probability* $\propto w_t^{[i]}$
9: *add* $x_t^{[i]}$ *to* $\mathcal{X}_t$
10: *endfor*
11: *return* $\mathcal{X}_t$

Re-sampling step

Monte Carlo Localization

- Each particle is a pose hypothesis
- Proposal is the motion model

$$x_t^{[j]} \sim p(x_t \mid x_{t-1}, u_t)$$

- Correction via the observation model

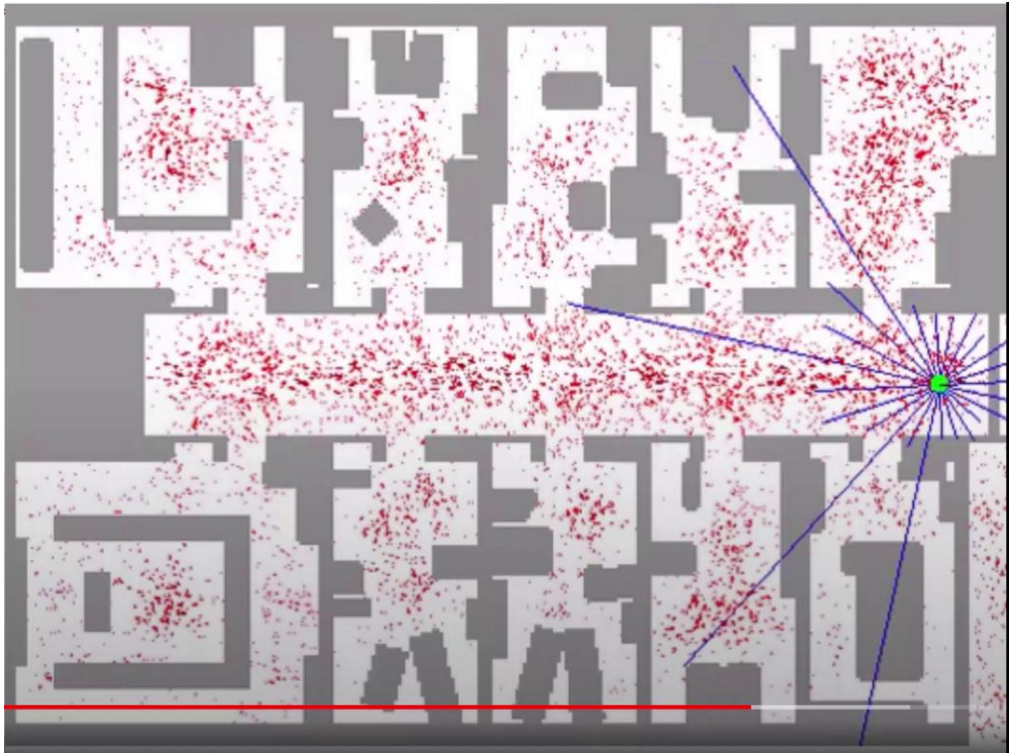
$$w_t^{[j]} = \frac{target}{proposal} \propto p(z_t \mid x_t, m)$$

Particle Filter for Localization

Particle_filter$(\mathcal{X}_{t-1}, u_t, z_t)$**:**

1: $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$
2: *for* $j = 1$ *to* J *do*
3: *sample* $x_t^{[j]} \sim p(x_t \mid u_t, x_{t-1}^{[j]})$
4: $w_t^{[j]} = p(z_t \mid x_t^{[j]})$
5: $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[j]}, w_t^{[j]} \rangle$
6: *endfor*
7: *for* $j = 1$ *to* J *do*
8: *draw* $i \in 1, \ldots, J$ *with probability* $\propto w_t^{[i]}$
9: *add* $x_t^{[i]}$ *to* $\mathcal{X}_t$
10: *endfor*
11: *return* $\mathcal{X}_t$

Particle Filter Example



Graph-based SLAM

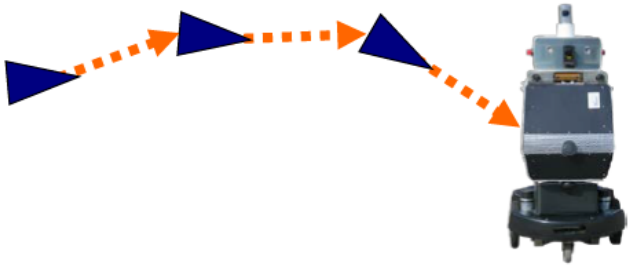
SLAM overview

Generally 2 different approaches:

- Filtering-based
  - Use EKF or Particle Filter for state estimation and build the map based on that estimation
  - Examples are GMapping, Hector SLAM
- Graph-based
  - Use pose graphs to store constraints between pose estimations
  - Use optimization for loop-closure to correct for pose estimations
  - Examples are KartoSLAM and Cartographer

Graph-based SLAM

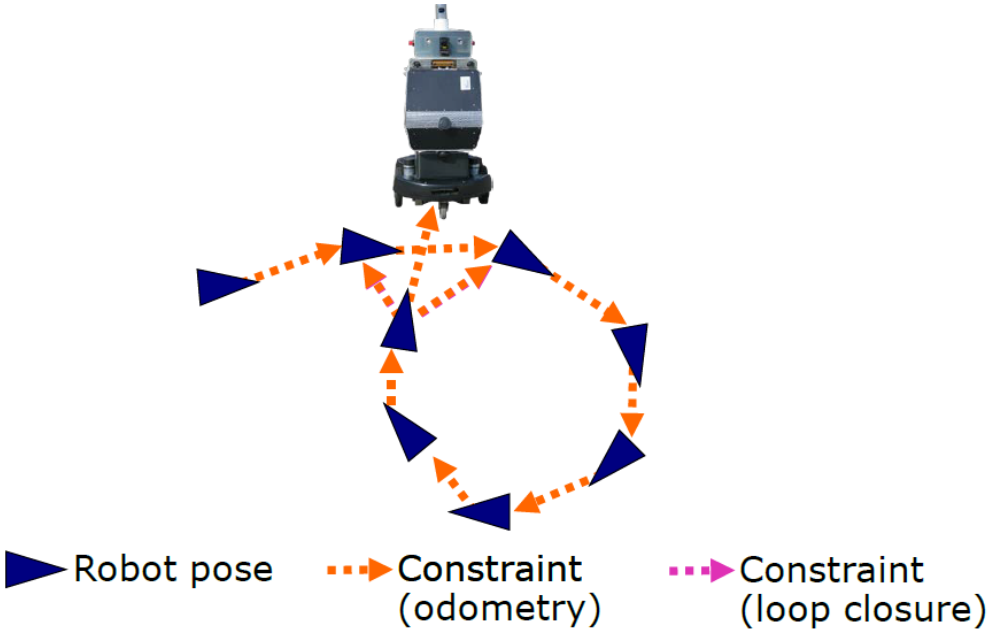
- Constraints connect the poses of the robot while it is moving
- Constraints are inherently uncertain



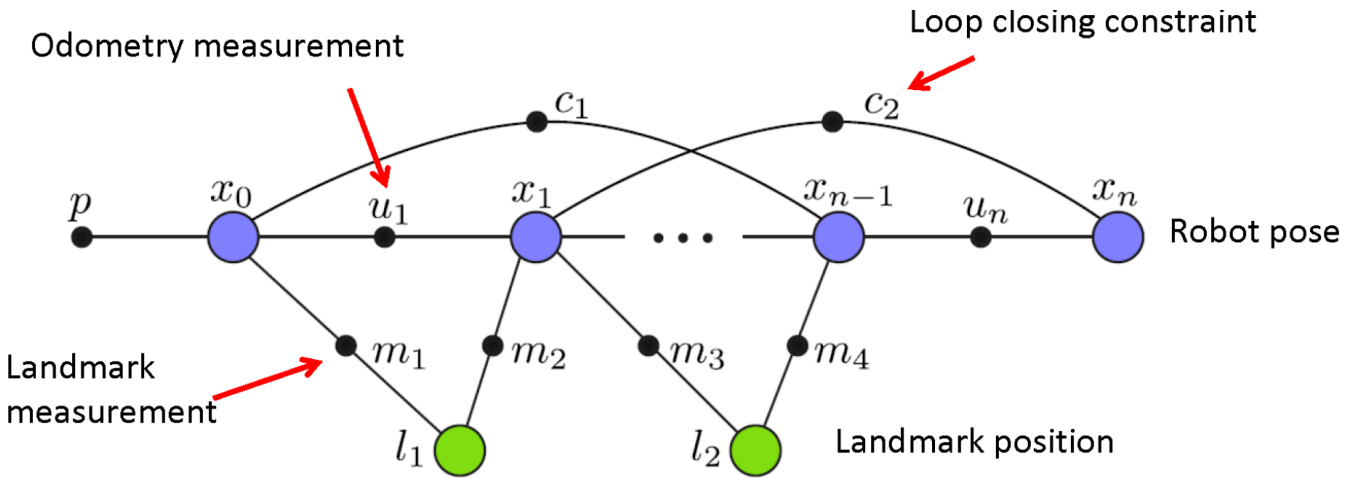
Robot pose Constraint

Graph-based SLAM

- Observing previously seen areas generates constraints between non-successive poses



Graph-based SLAM



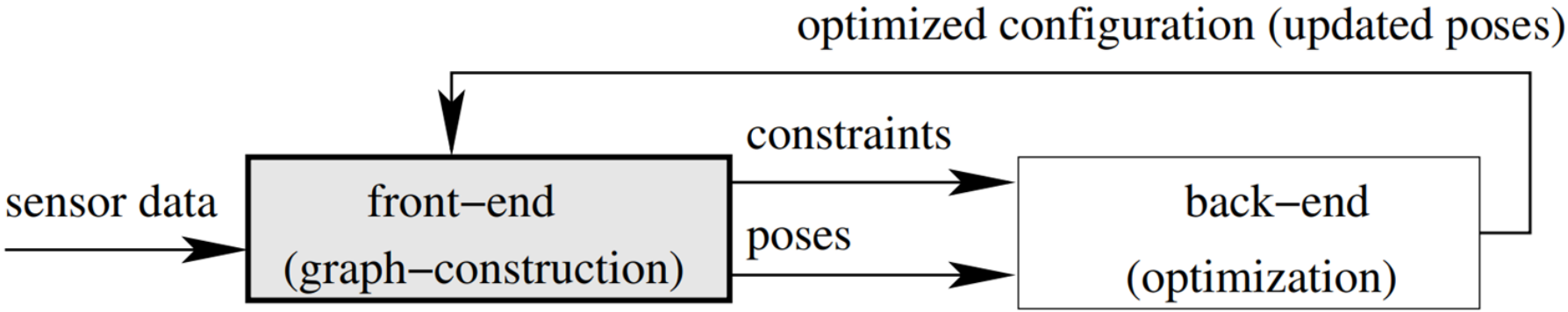
Bipartite graph with ***variable nodes*** and ***factor nodes***

General Idea of Graph-based SLAM

- Use a **graph** to represent the problem
- Every **node** in the graph corresponds to a pose of the robot during mapping
- Every **edge** between two nodes corresponds to a spatial constraint between them
- **Graph-Based SLAM:** Build the graph and find a node configuration that minimize the error introduced by the constraints (e.g., solving a least squares problem)

The Overall Graph-based SLAM System

- Interplay of front-end and back-end
- A consistent map helps to determine new constraints by reducing the search space



Some Notes

Graph-based SLAM is sensor agnostic. Only requires 2 types of sensors:

- One that can measure **relative** transformations between poses (IMU, odometry, wheel encoders, etc.). This sensor is used to build constraints between nodes.
- One that can measure **absolute** observation of the environment. (LiDARs, cameras, etc.). This sensor is used for loop closure

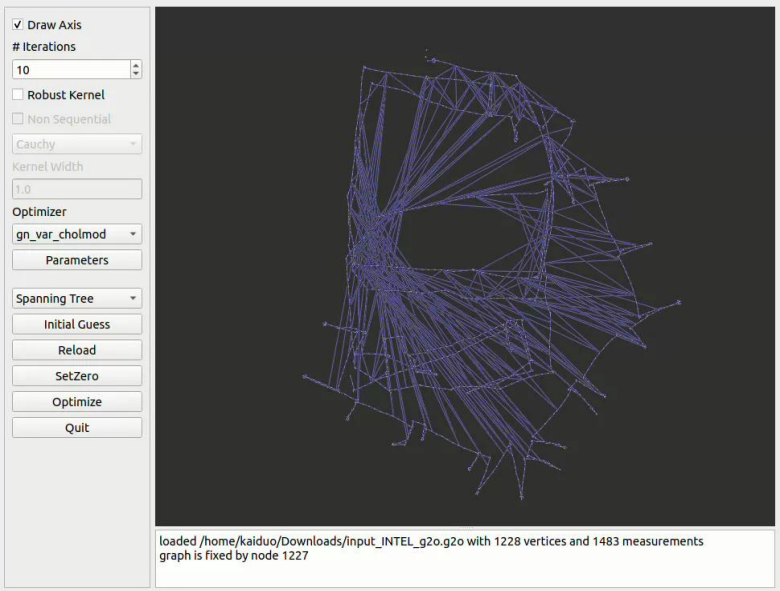
Some Notes

Graph-based SLAM is also scan (sensor) matching method agnostic.

- ICP
- Scan-to-scan
- Scan-to-map
- Map-to-map
- Feature-based
- RANSAC for outlier rejection
- Bundle Adjustment
- Correlative Scan Matching

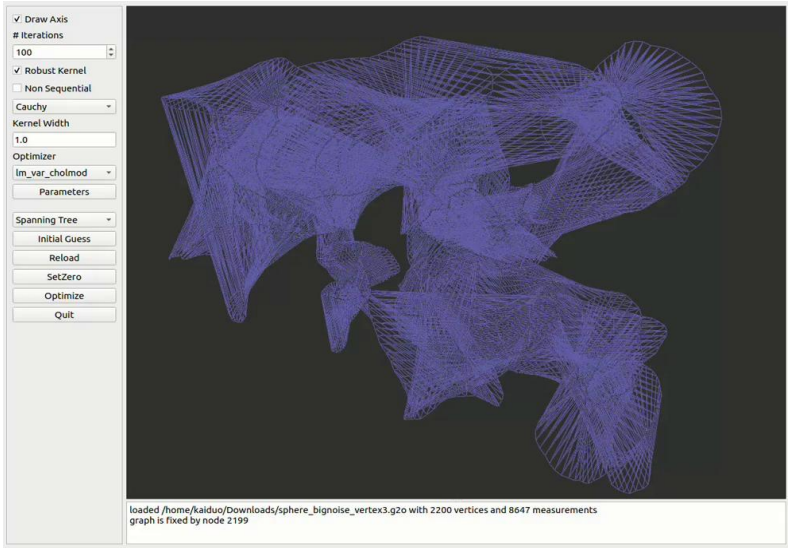
Graph-based SLAM Example (1)

- Solving nonlinear least square problem
  - Optimizing the Intel 2D graph is simple and fast



Graph-based SLAM Example (2)

- Solving nonlinear least squares problem.
  - Optimizing the graph with big noises and large number of variables is really hard.
  - The following video (4x speed) is a simulated sphere trajectory, corrupted by big noises.



Questions?